TABLE OF CONTENTS

1. INTRODUCTION 7

1.1 Background and Motivation… 7

1.2 Problem Statement…… 9

1.3 Objectives……11

1.4 Scope of the Project……12

2. LITERATURE REVIEW 14

2.1 Overview of Robotics in Fire Safety 14

2.2 Existing Fire Fighting Systems 16

2.3 Arduino and ESP32 Platforms 18

3. SYSTEM COMPONENTS AND HARDWARE 20

3.1 ESP32 Microcontroller 20

3.2 Motor Driver Integration 24

3.3 Flame Sensors 26

3.4 DC Motors and Wheels 29

3.5 Water Pump System 32

3.6 Servo Motor 35

3.7 Buck Converter 37

3.8 Power Supply 38

4. SYSTEM DESIGN AND ARCHITECTURE 41

4.1 Block Diagram 41

4.2 Circuit Design 44

4.3 Mechanical Structure 47

5. WORKING PRINCIPLE 51

5.1 Fire Detection Mechanism 51

5.2 Navigation Algorithm 53

5.3 Extinguishing Process 55

6. SOFTWARE IMPLEMENTATION 58

6.1 Arduino IDE 58

6.2 Programming Logic 60

6.3 Code Explanation 63

7. MANUAL MODE - BLUETOOTH CONTROL.....65

7.1 Bluetooth Module HC-05.....65

7.2 RC Controller App 66

7.3 Manual Control Implementation 68

8. AUTOMATIC MODE - TELEGRAM INTEGRATION 70

8.1 ESP32 WiFi Capabilities 70

8.2 Telegram Bot API..... 71

8.3 Mission Flow.....73

8.4 Automatic Mode Implementation.....75

9. 3D PRINTED CHASSIS..... 77

9.1 SolidWorks Chassis Design.....77

9.2 3D Printing in PLA 79

10. APPLICATIONS..... 82

10.1 Industrial Applications 82

10.2 Commercial Applications 83

10.3 Educational Applications.....84

11. ADVANTAGES AND LIMITATIONS.....86

11.1 Advantages 86

11.2 Limitations.....88

11.3 Mitigation Strategies 90

12. FUTURE ENHANCEMENTS 92

12.1 Sensor Upgrades.....92

12.2 Communication Capabilities 94

12.3 Intelligence and Automation 95

12.4 Mechanical Improvements 96
12.5 Integration with Smart Systems 98
13. CONCLUSION.....99
13.1 Summary of Achievements 99
13.2 Key Learnings 100
13.3 Significance 101
13.4 Future Outlook 102
13.5 Final Remarks……103
REFERENCES……...104

1. INTRODUCTION

1.1 Background and Motivation

Fire accidents pose a significant threat to human life and property across the globe. According to the National Fire Protection Association (NFPA), fire departments respond to a fire every 23 seconds in the United States alone. The statistics are equally alarming in developing countries where fire safety infrastructure is often inadequate. Traditional firefighting methods, while effective, frequently put firefighters in life-threatening situations, especially in hazardous environments such as chemical plants, nuclear facilities, and large industrial complexes.

The global impact of fire-related incidents extends far beyond immediate property damage. Each year, fires claim thousands of lives worldwide, leaving countless families devastated and communities shattered. The economic toll is staggering, with billions of dollars lost annually due to fire damage, business interruptions, and insurance claims. Beyond the tangible losses, fires create long-lasting psychological trauma for survivors and witnesses, affecting mental health and community well-being for years after the incident.

The motivation behind developing autonomous firefighting systems stems from the need to protect human lives while effectively combating fires. Robotics and automation technologies have advanced significantly in recent years, making it possible to design machines that can perform dangerous tasks without risking human safety. The integration of microcontrollers, sensors, and actuators has opened new possibilities in creating intelligent firefighting systems that can operate in environments too dangerous for human responders.

The Fire Fighting Bot project represents a practical application of embedded systems, IoT communication, and robotics principles to address a real-world safety challenge. By combining flame detection sensors, motor control systems, water pumping mechanisms, and modern communication technologies like Telegram, this project demonstrates how relatively simple components can be integrated to create a functional autonomous system with dual-mode operation.

This approach showcases the potential for technology to augment human capabilities in emergency response scenarios.

Historical context reveals that the concept of automated firefighting dates back several decades, but only recent advances in microelectronics, sensor technology, and wireless communication have made practical implementations feasible. Early attempts at robotic firefighting were limited by bulky electronics, unreliable sensors, and primitive control systems. Today's microcontrollers pack more computing power into a few square millimetres than entire rooms of equipment from previous generations, enabling sophisticated autonomous behaviors in compact, affordable packages.

The development of this Fire Fighting Bot aligns with broader trends in automation and robotics for hazardous environments. Industries ranging from mining to nuclear power have increasingly turned to robotic systems to handle dangerous tasks, recognizing both the safety benefits and the operational advantages of machines that can work continuously without fatigue. Firefighting represents a particularly challenging application domain due to the unpredictable nature of fires, the harsh environmental conditions, and the critical importance of rapid, effective response.

Educational institutions worldwide have recognized the value of firefighting robot projects as comprehensive learning experiences. These projects integrate multiple engineering disciplines including mechanical design, electronics, programming, and systems integration. Students working on such projects gain practical skills while contributing to potentially life-saving technologies. The Fire Fighting Bot project follows in this tradition, providing both educational value and practical utility.

The dual-mode operation capability of this Fire Fighting Bot addresses a key challenge in robotic firefighting: the balance between autonomous operation and human oversight. Fully autonomous systems can respond immediately without waiting for human commands, but may struggle with complex situations requiring judgment and experience. Manual control allows human operators to apply their expertise, but introduces delays and puts operators at risk. By providing both

modes, the Fire Fighting Bot offers flexibility to adapt to different scenarios and operator preferences.

The choice of ESP32 as the primary microcontroller reflects careful consideration of the project's requirements. The ESP32's integrated Wi-Fi and Bluetooth capabilities eliminate the need for separate communication modules, reducing cost and complexity. Its dual-core processor provides sufficient computing power for sensor processing, motor control, and communication tasks simultaneously. The extensive GPIO capabilities accommodate all necessary sensor and actuator connections without requiring expansion hardware.

The integration of Telegram Bot API for automatic mode operation represents an innovative approach to remote control and monitoring. Telegram provides a reliable, globally accessible messaging platform that requires no additional infrastructure beyond an internet connection. Users can trigger firefighting missions, receive status updates, and monitor progress from anywhere in the world using their smartphones. This cloud-based approach eliminates the range limitations of traditional radio control systems and enables new use cases such as remote facility monitoring.

1.2 Problem Statement

The primary problem addressed by this project is the need for an automated fire detection and extinguishing system that can operate in environments where human intervention is either dangerous or impractical. Traditional firefighting approaches face several significant challenges that limit their effectiveness and safety.

First and foremost, human firefighters face significant risks when entering burning buildings or hazardous environments. Smoke inhalation, structural collapse, extreme temperatures, and exposure to toxic chemicals are constant threats. Even with protective equipment, firefighters suffer injuries and fatalities at alarming rates. According to NFPA statistics, tens of thousands of firefighter injuries occur annually in the United States alone, with hundreds of line-of-duty deaths. Each injury or death represents not only a personal tragedy but also a loss of experienced personnel and a burden on emergency services resources.

Response time to fire incidents can be critical, especially in remote locations or facilities with limited staffing. Fires can double in size every minute under favorable conditions, meaning that delays of even a few minutes can transform manageable incidents into major disasters. Remote industrial facilities, offshore platforms, and automated warehouses may have no personnel present when fires start, allowing them to grow unchecked until external responders arrive. The Fire Fighting Bot addresses this challenge by providing on-site, immediate response capability.

Continuous monitoring of large facilities requires significant human resources that may not be economically feasible for many organizations. Security guards conducting periodic patrols cannot provide the constant vigilance needed for early fire detection. Fixed fire detection systems can identify fires but cannot take action to extinguish them without human intervention or expensive, complex suppression systems. The Fire Fighting Bot offers a middle ground: autonomous detection and response without the cost of full-time staffing.

Early detection and immediate response can prevent small fires from becoming major disasters. Statistics consistently show that fires extinguished in their initial stages cause minimal damage, while those allowed to grow result in catastrophic losses. The Fire Fighting Bot's continuous monitoring and rapid response capability addresses this need by detecting fires at their earliest stages and taking immediate action before they can spread.

Cost-effective fire safety solutions are needed for small and medium enterprises that cannot afford comprehensive fire suppression systems or dedicated fire safety personnel. Traditional sprinkler systems require expensive installation and maintenance. Professional fire monitoring services charge ongoing fees that strain limited budgets. The Fire Fighting Bot provides an affordable alternative that can be deployed incrementally and scaled as resources permit.

The Fire Fighting Bot aims to address these challenges by providing a low-cost, dual-mode solution that can detect fires early and take immediate action to extinguish them before they spread. The automatic mode with Telegram integration allows for remote mission triggering, while the manual mode provides

precise control in complex scenarios. This flexibility ensures that the system can adapt to different operational requirements and user capabilities.

Beyond the immediate technical challenges, the project addresses broader issues of technology accessibility and democratization of advanced safety systems. By using widely available components and open-source platforms, the Fire Fighting Bot demonstrates that sophisticated automation need not be limited to well-funded organizations. Students, hobbyists, and small businesses can build, modify, and deploy similar systems, spreading the benefits of robotic firefighting more broadly than proprietary commercial solutions.

1.3 Objectives

The main objectives of this project are comprehensive and address multiple aspects of firefighting robot design and implementation. Each objective contributes to the overall goal of creating a functional, affordable, and educational firefighting system.

1. To design a robust chassis using SolidWorks and manufacture it using 3D printing technology with PLA material. The chassis design must accommodate all components while maintaining stability, durability, and ease of assembly.
2. To implement a dual-mode operation system: Manual mode via Bluetooth RC app and Automatic mode via Telegram-triggered missions. Both modes must function reliably and provide appropriate capabilities for their intended use cases.
3. To integrate flame sensors for directional fire detection and servo motor for precise nozzle positioning. The sensor system must reliably detect fires and provide sufficient information for navigation and targeting.
4. To create an extinguishing mechanism using a water pump system with adequate power supply management using buck converter. The extinguishing system must deliver sufficient water to suppress fires while managing power efficiently.
5. To demonstrate the practical application of embedded systems and IoT in solving real-world fire safety problems. The completed project should serve as

both a functional tool and an educational example of modern engineering practices.

1.4 Scope of the Project

The scope of this Fire Fighting Bot project encompasses the design, development, and testing of a prototype autonomous firefighting robot with dual-mode capabilities. The project boundaries have been carefully defined to ensure achievable goals while maintaining meaningful outcomes.

The hardware design and assembly include the ESP32 microcontroller and integrated motor driver. The hardware must provide reliable operation under expected conditions while maintaining compact size and reasonable cost.

The 3D modelling of chassis using SolidWorks and manufacturing via 3D printing in PLA material represents a significant portion of the mechanical design effort. The chassis must accommodate all electronic components, motors, sensors, and the water container while maintaining appropriate weight distribution and structural integrity.

Integration of flame detection sensors for fire sensing and servo motor for nozzle control requires careful consideration of sensor placement, wiring, and software algorithms. The sensor system must provide reliable detection across expected operating conditions while minimizing false positives.

Implementation of Bluetooth communication for manual mode control via smartphone app involves selecting appropriate hardware, configuring communication protocols, and developing the control interface. The manual mode must provide responsive, intuitive control for operators.

Development of Telegram Bot API integration for automatic mode mission triggering represents the project's IoT component. This includes bot creation, API integration, message handling, and secure communication implementation.

Implementation of water pump control with buck converter for voltage regulation ensures that the extinguishing system operates reliably while managing power

efficiently. The power system must provide stable voltages to all components while maximizing battery life.

Programming of both Arduino Uno (manual mode) and ESP32 (automatic mode) for coordinated operation involves developing, testing, and debugging software for both microcontrollers. The software must implement all required functionality while maintaining reliability and responsiveness.

Testing and validation of both manual and automatic modes ensures that the system operates as intended under various conditions. This includes functional testing, performance evaluation, and identification of areas for improvement.

The modular design allows for future enhancements such as camera integration, advanced navigation algorithms, cloud-based monitoring, and integration with building management systems. While these enhancements are outside the current project scope, the design accommodates them for future development.

2. LITERATURE REVIEW

2.1 Overview of Robotics in Fire Safety

The application of robotics in fire safety has evolved significantly over the past few decades, transforming from simple remote-controlled devices to sophisticated autonomous systems. This evolution reflects broader advances in robotics, artificial intelligence, and sensor technology, as well as growing recognition of the value that robots can provide in hazardous environments.

Early firefighting robots were primarily teleoperated devices that allowed firefighters to approach dangerous situations while maintaining a safe distance. These systems provided valuable capabilities but still required significant human intervention and specialized training to operate effectively. The robots typically featured basic cameras and sensors that relayed information to operators, who made decisions about navigation and extinguishing actions. While limited by communication delays and operator skill, these early systems demonstrated the potential for robotics in firefighting applications.

Modern firefighting robots incorporate advanced sensors, artificial intelligence, and autonomous navigation capabilities. Research in this field has focused on several key areas including fire detection algorithms, path planning in smoke-filled environments, heat-resistant materials, and effective extinguishing mechanisms. The integration of these technologies has produced robots capable of operating with minimal human oversight in complex, dynamic environments.

A study by Murphy et al. (2012) surveyed the state of the art in rescue robots and identified key challenges including communication in harsh environments, mobility over debris, and sensor reliability in high-temperature conditions. The research highlighted the importance of robust design and redundant systems for mission-critical applications. Communication systems must function despite interference from smoke, flames, and structural materials. Mobility systems must navigate obstacles, stairs, and uneven terrain while carrying heavy extinguishing equipment. Sensors must provide accurate data despite heat, smoke, and water exposure.

Japanese researchers have been particularly active in this field, developing several advanced firefighting robots following the Kobe earthquake. The disaster revealed gaps in emergency response capabilities and motivated investment in robotic systems. The Tokyo Fire Department has deployed various robotic systems including the T-52 Enryu, which can spray water at rates up to 4,000 liters per minute. These large-scale systems demonstrate the potential of robotic firefighting but also highlight the cost and complexity barriers that limit widespread adoption.

European research has focused on collaborative robotic systems that work alongside human firefighters rather than replacing them entirely. The GUARDIANS project developed robot swarms for search and rescue in hazardous environments, demonstrating how multiple simple robots can achieve complex objectives through coordination. This approach offers advantages in robustness and scalability compared to single, complex robots.

American research has emphasized mobility and manipulation capabilities, with projects like the DARPA Robotics Challenge pushing the boundaries of what robots can do in disaster scenarios. While not specifically focused on firefighting, these programs have produced technologies applicable to emergency response, including advanced locomotion systems, dexterous manipulation, and human-robot interaction interfaces.

Academic research has explored various technical approaches to firefighting robot design. Machine learning algorithms for fire detection have achieved high accuracy in laboratory conditions, though real-world performance remains challenging due to variable lighting, smoke, and fire characteristics. Navigation algorithms for smoke-filled environments have progressed from simple obstacle avoidance to sophisticated simultaneous localization and mapping (SLAM) systems that build maps while navigating. Extinguishing mechanisms have evolved from simple water sprays to targeted systems that optimize agent delivery based on fire characteristics.

Commercial development has produced several firefighting robot products, though adoption remains limited by cost and operational complexity. Companies like Howe and Howe Technologies, Shark Robotics, and Milrem have developed

robots for military and civilian firefighting applications. These systems typically cost hundreds of thousands of dollars, placing them beyond the reach of most fire departments and industrial facilities.

The literature reveals a gap between advanced research systems and affordable, practical solutions. While research robots demonstrate impressive capabilities, their complexity and cost limit real-world deployment. Meanwhile, simpler systems that could provide immediate value remain underdeveloped. The Fire Fighting Bot project addresses this gap by demonstrating that effective firefighting robots can be built using affordable, widely available components.

2.2 Existing Fire Fighting Systems

Several categories of firefighting robots exist in the market today, ranging from simple remote-controlled units to sophisticated autonomous systems. Understanding these categories helps position the Fire Fighting Bot project within the broader landscape of firefighting technology.

2.2.1 Teleoperated Systems

Teleoperated firefighting robots are controlled remotely by human operators who view the environment through cameras and sensors. These systems typically provide feedback to the operator, who makes decisions about navigation and extinguishing. While effective in many scenarios, these systems require trained operators and continuous communication links. The operator's skill and situational awareness directly impact system performance.

Examples of teleoperated systems include the Thermite RS1-T2, a compact robot designed for hazardous material incidents and structural fires. It features a tracked drive system for mobility over debris, a high-definition camera for operator visibility, and a water cannon capable of delivering 600 gallons per minute. The robot's small size allows it to enter spaces too dangerous or confined for human firefighters.

The Colossus robot, deployed by the Paris Fire Brigade, gained international attention during the Notre Dame Cathedral fire. This teleoperated system carried

water hoses into the burning structure, protecting firefighters from collapsing elements and extreme heat. Its deployment demonstrated the value of robotic systems in protecting both property and human life.

2.2.2 Semi-Autonomous Systems

Semi-autonomous systems combine human oversight with automated functions, offering a balance between automation and human judgment. For example, the robot may autonomously navigate to a fire location while the operator controls the extinguishing mechanism. This approach leverages the strengths of both human operators and automated systems.

Navigation autonomy reduces the cognitive load on operators, allowing them to focus on firefighting decisions rather than low-level robot control. Automated navigation systems can process sensor data faster than humans, potentially identifying optimal paths that operators might miss. However, human oversight remains important for handling unexpected situations and making strategic decisions.

The SAFFiR (Shipboard Autonomous Firefighting Robot) project developed by the U.S. Naval Research Laboratory represents an advanced semi-autonomous system. SAFFiR can navigate ships autonomously, locate fires using thermal imaging, and manipulate fire hoses under human supervision. The system demonstrates the feasibility of autonomous navigation in complex, dynamic environments.

2.2.3 Fully Autonomous Systems

Fully autonomous firefighting robots can detect fires, navigate to their location, and extinguish them without human intervention. These systems require sophisticated sensors, advanced algorithms, and reliable decision-making capabilities. While more complex to develop, they offer the greatest potential for protecting human lives by eliminating the need for human presence in dangerous environments.

Autonomous systems must handle the complete firefighting workflow: continuous monitoring for fire detection, path planning to reach detected fires, positioning for

effective extinguishing, and verification that fires are completely suppressed. Each of these tasks presents significant technical challenges that must be addressed through careful system design.

Research systems have demonstrated autonomous firefighting in controlled environments, but real-world deployment remains limited. The complexity of real fire scenarios, including unpredictable fire behaviour, structural instability, and communication challenges, has prevented widespread adoption of fully autonomous systems.

2.3 Arduino and ESP32 Platforms

The Arduino platform has revolutionized the field of embedded systems and DIY electronics, making microcontroller programming accessible to artists, designers, and hobbyists without extensive electronics backgrounds. Created in 2005 by Massimo Banzi and his team in Italy, Arduino was designed to simplify the creation of interactive electronic projects.

The Arduino Uno, based on the ATmega328P microcontroller, has become one of the most popular boards in the Arduino lineup. Its features include 14 digital I/O pins, 6 analog inputs, a 16 MHz quartz crystal, USB connection, power jack, and ICSP header. The board operates at 5V and can be powered via USB or external power supply. The simplicity and reliability of the Arduino Uno have made it a staple in educational institutions and maker communities worldwide.

The Arduino ecosystem includes a vast library of community-contributed code, extensive documentation, and a supportive user community. These resources enable beginners to learn quickly while providing advanced users with tools for complex projects. The standardized form factor and pin layout ensure compatibility with a wide range of shields and accessories.

The ESP32, developed by Espressif Systems, represents a significant advancement in IoT-capable microcontrollers. Key features include a dual-core Xtensa 32-bit LX6 microprocessor running at up to 240 MHz, integrated Wi-Fi (802.11 b/g/n) and Bluetooth (Classic and BLE), 520 KB SRAM and 448 KB ROM, 34 programmable GPIO pins, multiple analog-to-digital converters

(ADCs), hardware acceleration for cryptographic functions, and ultra-low power consumption modes.

The ESP32's integrated wireless capabilities eliminate the need for separate communication modules, reducing both cost and complexity. The Wi-Fi capability enables internet connectivity for cloud services, remote monitoring, and over-the-air updates. Bluetooth support allows connection to smartphones, tablets, and other devices for local control and configuration.

For the Fire Fighting Bot project, the ESP32 provides the processing power, Wi-Fi connectivity for Telegram integration, and sufficient I/O capabilities to handle flame sensor inputs, motor control outputs, servo positioning, and pump activation. Its integrated wireless capabilities eliminate the need for separate communication modules, simplifying the design and reducing costs.

The ESP32 development community has grown rapidly, producing extensive libraries, tutorials, and example projects. The Arduino IDE support for ESP32 makes it accessible to users familiar with Arduino programming, while the ESP-IDF framework provides advanced capabilities for experienced developers. This combination of power and accessibility makes the ESP32 an ideal choice for projects requiring both local processing and wireless connectivity.

Comparative analysis of available platforms reveals the ESP32's advantages for this application. While other microcontrollers offer specific strengths, none match the ESP32's combination of processing power, wireless integration, I/O capabilities, and cost effectiveness. The STM32 family offers similar processing power but lacks integrated wireless. The Raspberry Pi provides more computing power but consumes more power and requires more complex software. The ESP32 hits the sweet spot for battery-powered IoT applications like the Fire Fighting Bot.

3. SYSTEM COMPONENTS AND HARDWARE

The Fire Fighting Bot consists of several key hardware components that work together to detect and extinguish fires. This chapter provides detailed information about each component, its specifications, and its role in the system. Understanding each component's function and characteristics is essential for building, troubleshooting, and modifying the Fire Fighting Bot.

3.1 ESP32 Microcontroller

The ESP32 serves as the central processing unit for the automatic mode operation of the Fire Fighting Bot. It is a powerful System-on-Chip (SoC) that integrates Wi-Fi and Bluetooth connectivity along with a dual-core processor, making it an ideal choice for IoT applications requiring both local processing and wireless communication.

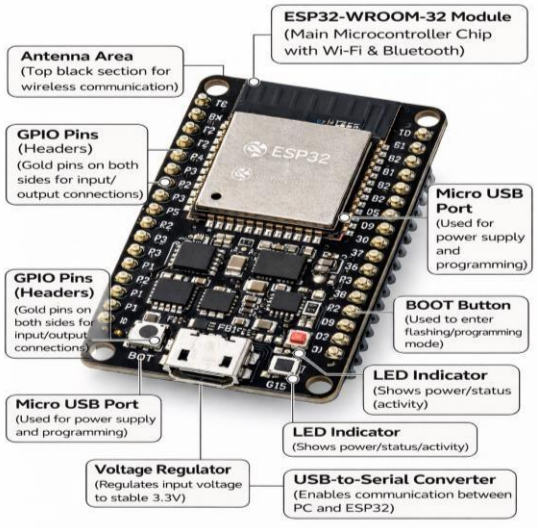


FIG 1. ESP32-WROOM-32 Module

3.1.1 Technical Specifications

The ESP32-WROOM-32 module used in this project offers impressive specifications that enable the Fire Fighting Bot's advanced capabilities. The dual-core Xtensa 32-bit LX6 microprocessor can operate at frequencies up to 240 MHz, providing ample computing power for real-time sensor processing, motor

control, and wireless communication simultaneously. This processing capability ensures responsive operation even when multiple tasks are executing concurrently.

The 520 KB of SRAM provides sufficient memory for program variables, sensor data buffers, and communication stacks. While this may seem modest compared to general-purpose computers, it is generous for microcontroller applications and accommodates complex algorithms without memory constraints. The 448 KB of ROM contains bootloader code and other fixed data, while the typical 4 MB of flash memory stores the application program and persistent data.

The integrated Wi-Fi radio supports 802.11 b/g/n protocols in the 2.4 GHz band, enabling connection to standard wireless networks. This capability is essential for the Telegram integration feature, allowing the Fire Fighting Bot to communicate with cloud services from anywhere with internet access. The Wi-Fi implementation includes power-saving modes that extend battery life during standby operation.

Bluetooth v4.2 support includes both Classic Bluetooth for legacy device compatibility and Bluetooth Low Energy (BLE) for power-efficient communication. While the current implementation primarily uses Wi-Fi for automatic mode, the Bluetooth capability enables future expansion for local device pairing and configuration.

The 34 programmable GPIO pins provide flexible connectivity for sensors, actuators, and other peripherals. These pins support various functions including digital I/O, analog input, PWM output, and communication protocols (I2C, SPI, UART). The abundance of GPIO pins eliminates the need for expansion hardware, simplifying the design and reducing cost.

The 18 ADC channels with 12-bit resolution enable precise analog sensor readings. This resolution provides 4096 distinct values, sufficient for detecting subtle changes in sensor outputs. The multiple channels accommodate the three flame sensors plus additional sensors for future expansion.

The 2 DAC channels with 8-bit resolution enable analog output for applications requiring precise voltage control. While not used in the current implementation,

these channels provide expansion capability for future enhancements such as analog motor control or audio feedback.

Operating voltage of 3.0V to 3.6V (typically 3.3V) is compatible with modern sensors and logic devices. The wide temperature range of -40°C to +85°C ensures reliable operation in various environmental conditions, though the current Fire Fighting Bot design is not intended for extreme temperature exposure.

Parameter	Specification
Microprocessor	Dual-core Xtensa 32-bit LX6
Clock Speed	Up to 240 MHz
SRAM	520 KB
ROM	448 KB
Flash Memory	4 MB (typical)
Wi-Fi	802.11 b/g/n (2.4 GHz)
Bluetooth	v4.2 BR/EDR and BLE
GPIO Pins	34 programmable
ADC Channels	18 channels (12-bit resolution)
DAC Channels	2 channels (8-bit)
Operating Voltage	3.0V - 3.6V (typically 3.3V)
Operating Temperature	-40°C to +85°C

3.1.2 Role in the Fire Fighting Bot

In the Fire Fighting Bot system, the ESP32 performs multiple critical functions that enable autonomous operation. Each function leverages specific hardware capabilities of the ESP32 to achieve reliable, responsive performance.

The ESP32 connects to Wi-Fi network for Telegram Bot API communication, enabling remote mission triggering and status reporting. This connection requires

proper Wi-Fi credentials and internet access. The ESP32 handles all aspects of Wi-Fi connectivity including network scanning, authentication, and maintaining the connection. Automatic reconnection ensures robust operation even if the connection is temporarily lost.

Receiving mission commands via Telegram messages involves polling the Telegram Bot API for new messages, parsing command text, and executing appropriate actions. The ESP32 formats HTTP requests, sends them to Telegram servers, and processes the JSON responses. This communication occurs over the Wi-Fi connection, with the ESP32 acting as an HTTP client.

Reading analog values from flame sensors uses the ESP32's ADC capabilities. The analogRead() function converts sensor voltages to digital values that can be processed by the software. Regular sampling ensures timely fire detection while managing power consumption. The 12-bit resolution provides sufficient sensitivity to detect fires at practical distances.

Processing sensor data to determine fire location and direction involves comparing readings from multiple sensors and applying decision logic. The ESP32's processing power enables real-time analysis without significant delays. The results drive navigation decisions that guide the robot toward detected fires.

Controlling DC motors through the integrated motor driver requires generating appropriate digital and PWM signals. The ESP32's GPIO pins provide the necessary outputs, while the PWM capability enables speed control. Precise timing ensures smooth motor operation and accurate navigation.

Positioning the water nozzle using servo motor control leverages the ESP32's PWM generation capability. Servo motors require specific pulse widths to achieve desired positions, and the ESP32's hardware PWM provides accurate, jitter-free control. The servo sweeps across its range to find optimal aiming positions.

Activating the water pump via relay module uses a simple digital output controlled by the ESP32. When fire is detected and the robot is properly positioned, the ESP32 sets the control pin HIGH to activate the relay, which completes the pump circuit. Safety timeouts prevent excessive pump operation.

Implementing the autonomous control logic for automatic mode operation ties all these functions together into a coherent behavior. The ESP32 executes the state machine that governs robot behavior, transitioning between states based on sensor inputs and mission progress. This logic ensures systematic fire detection, approach, and extinguishing.

3.2 Motor Driver Integration

The motor driver circuit is integrated directly onto the BOARD, providing compact and efficient motor control. The design is based on the L298N dual H-bridge configuration, allowing independent control of two DC motors. This integration reduces wiring complexity and improves reliability compared to discrete motor driver modules.

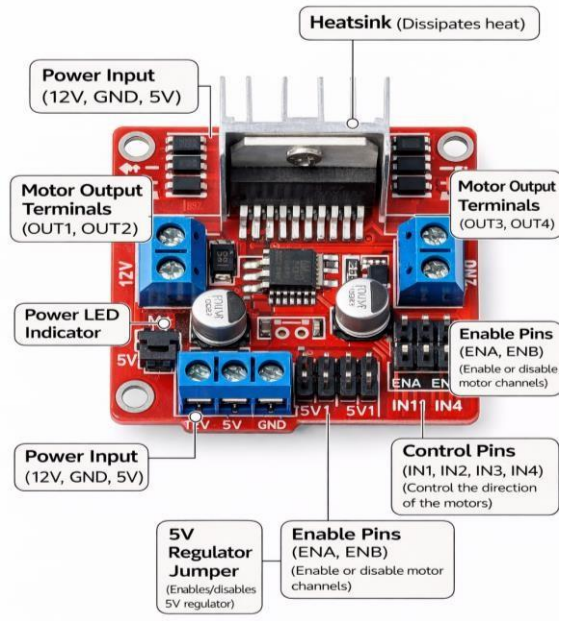


FIG 2. L298N Motor Driver

3.2.1 Technical Specifications

The integrated motor driver provides robust performance for the Fire Fighting Bot's locomotion requirements. The motor supply voltage range of 5V to 35V accommodates various motor types and power supplies. This flexibility allows the use of common 12V motors while supporting other configurations as needed.

The maximum motor supply current of 2A per channel provides sufficient drive for typical DC motors used in robotics applications. This current capacity handles normal operation while providing headroom for startup surges and brief overloads. Over-current protection prevents damage from stalled motors or short circuits.

The logic level voltage range of 3.3V to 5V ensures compatibility with both the ESP32's 3.3V logic and the 5V logic used by some sensors and modules. This dual-voltage capability simplifies interfacing with various components without requiring level translation.

The maximum power dissipation of 25W requires appropriate thermal management during continuous operation. Heat sinks or cooling fans may be necessary for high-duty-cycle applications. The BOARD design includes copper area for heat spreading to manage thermal performance.

The 3.3V/5V TTL/CMOS compatible control signals interface directly with the ESP32 GPIO pins without additional circuitry. This direct connection reduces component count and potential failure points.

Built-in protection features include over-temperature shutdown to prevent damage from excessive heat, and over-current protection to limit current during faults. These protections improve reliability and prevent catastrophic failures.

Parameter	Specification
Motor Supply Voltage	5V to 35V
Motor Supply Current	Up to 2A per channel
Logic Level Voltage	3.3V to 5V
Maximum Power Dissipation	25W
Control Signals	3.3V/5V TTL/CMOS compatible
Protection	Over-temperature, over-current
Number of Channels	2 (dual H-bridge)

3.2.2 H-Bridge Operation

An H-bridge is an electronic circuit that enables a voltage to be applied across a load in either direction. It consists of four switching elements (typically transistors or MOSFETs) arranged in an H configuration. By controlling which switches are closed, the direction of current flow through the motor can be reversed, thereby controlling the motor's direction of rotation.

The name "H-bridge" derives from the circuit's schematic appearance, with the motor forming the crossbar of the H and the switches forming the vertical elements. This arrangement allows four distinct switch combinations that produce different motor behaviors.

When the top-left and bottom-right switches are closed, current flows through the motor in one direction, causing forward rotation. When the top-right and bottom-left switches are closed, current flows in the opposite direction, causing reverse rotation. When both top or both bottom switches are closed, the motor terminals are shorted together, creating a braking effect. When all switches are open, the motor coasts freely.

The integrated motor driver contains two H-bridges, allowing independent control of two DC motors. Each motor can be controlled in three states: forward rotation, reverse rotation, and stopped (brake or coast). This independent control enables differential drive steering, where different motor speeds and directions produce various movement patterns.

PWM signals applied to the enable pins control motor speed by rapidly switching the H-bridge on and off. The duty cycle of the PWM signal determines the average voltage applied to the motor, which in turn determines speed. Higher duty cycles produce higher speeds, while lower duty cycles produce lower speeds.

3.3 Flame Sensors

Flame sensors are critical components of the Fire Fighting Bot, as they enable the system to detect the presence and direction of fire accurately. These sensors work on the principle of detecting infrared (IR) radiation, which is strongly emitted by flames during combustion. The project utilizes IR-based flame sensors that are

highly sensitive to specific wavelengths of light, typically in the range of 760 nm to 1100 nm.

When a flame is present, the sensor captures the IR radiation and converts it into an electrical signal. This signal is then processed by the microcontroller to determine the intensity and direction of the fire source. Multiple flame sensors are placed at different angles on the robot to improve detection coverage and accuracy. The system can distinguish between normal light sources and actual flames based on the intensity and frequency of the IR signals.

Once a flame is detected, the bot can automatically navigate towards the fire location. This enables quick response and effective fire extinguishing action. Overall, flame sensors play a vital role in ensuring the efficiency and reliability of the Fire Fighting Bot system.

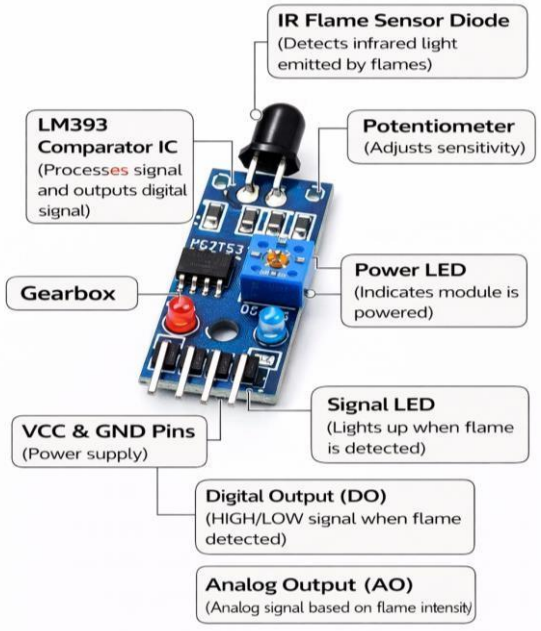


FIG 3. IR Flame Sensor Module

3.3.1 Working Principle

Flame sensors work based on the principle that fire emits infrared radiation in a specific wavelength range (approximately 760 nm to 1100 nm). This radiation is invisible to the human eye but can be detected by specialized photodetectors. The sensor contains a photodiode that is sensitive to this wavelength range.

When infrared radiation from a flame falls on the photodiode, it generates a small electrical current proportional to the intensity of the radiation. This photoelectric effect converts light energy into electrical energy, providing a measurable signal that indicates fire presence.

The sensor module typically includes an operational amplifier that amplifies the small signal from the photodiode to a usable level. The amplifier may also filter the signal to reduce noise and improve detection reliability. The gain and filtering characteristics affect the sensor's sensitivity and response time.

The output can be either analog (providing a voltage proportional to flame intensity) or digital (providing a high/low signal when flame is detected above a threshold). Analog outputs provide more information about fire intensity and distance, while digital outputs simplify processing by providing a clear detection indication.

The sensor's field of view determines the angular range over which it can detect fires. A narrower field of view provides better directional resolution but requires more sensors for complete coverage. A wider field of view provides broader coverage with fewer sensors but less precise direction indication.

3.3.2 Sensor Placement

For effective fire detection, multiple flame sensors are positioned at different angles on the robot. A typical configuration uses three sensors arranged to provide overlapping coverage of the forward hemisphere. This arrangement allows the robot to determine the direction of the fire and navigate accordingly.

The Left Sensor is angled 30-45 degrees to the left of the robot's centerline. This positioning enables detection of fires approaching from the left side. When the left sensor shows the strongest detection, the robot knows to turn left to center the fire in its path.

The Center Sensor points straight ahead, providing detection directly in front of the robot. This sensor is crucial for final approach to the fire, ensuring the robot moves directly toward the target. When the center sensor shows the strongest detection, the robot moves forward.

The Right Sensor is angled 30-45 degrees to the right of the robot's centerline, mirroring the left sensor. This completes the triad, enabling detection of fires on the right side. When the right sensor shows the strongest detection, the robot turns right.

By comparing the readings from all three sensors, the control algorithm can determine whether to turn left, turn right, or move forward. If multiple sensors detect fire, the direction with the strongest signal indicates the fire's location. This simple but effective approach enables reliable fire tracking without complex sensor arrays.

Parameter	Specification
Detection Range	760-1100 nm (infrared)
Detection Angle	60 degrees
Operating Voltage	3.3V – 5V
Output Type	Analog and Digital
Detection Distance	Up to 1 meter
Response Time	Less than 1 ms

3.4 DC Motors and Wheels

DC motors provide the locomotion for the Fire Fighting Bot, enabling it to move in different directions and navigate toward detected fires. The project uses 12V or 5V DC motors equipped with reduction gearboxes to increase torque and ensure smooth movement. These motors help the robot navigate various surfaces efficiently while carrying the weight of all components and the water supply.

The speed and direction of the motors are controlled by a motor driver circuit connected to the microcontroller. This allows the bot to move forward, backward, and turn accurately while approaching the fire source. Precise motor control is essential for effective navigation, especially when making fine adjustments to position the robot for extinguishing.

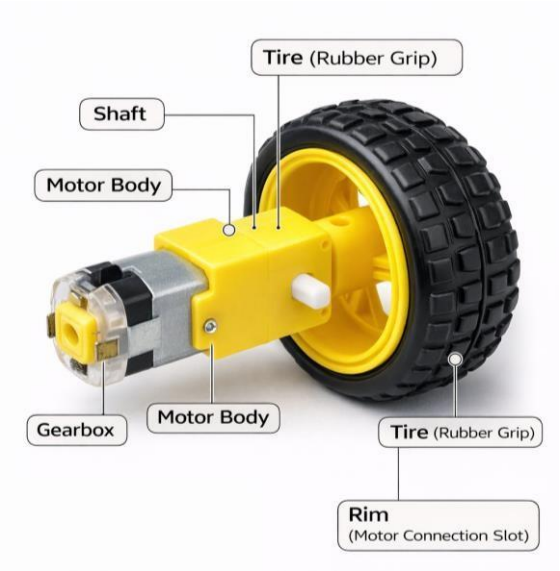


FIG 4. DC Motor and Wheels

3.4.1 Motor Specifications

The DC motors selected for the Fire Fighting Bot balance performance, cost, and availability. The 12V or 5V operating voltage is compatible with common battery configurations and power supplies. Lower voltage motors may be preferred for battery-powered operation to reduce the number of cells required.

The 100-300 RPM speed range (after gearbox reduction) provides appropriate movement speed for firefighting applications. Higher speeds enable rapid response but may reduce precision and increase stopping distance. Lower speeds provide better control but increase response time. The selected range balances these considerations.

The torque output must be sufficie”t to’move the robot chassis and water container on expected surfaces. The gearbox reduction increases torque at the expense of speed, providing the mechanical advantage needed for acceleration and hill climbing. Actual torque requirements depend on robot weight, wheel diameter, and surface conditions.

Normal load current of 0.5A to 1A indicates efficient operation under typical conditions. This current range is manageable for standard motor drivers and power supplies. Stall current of 2A to 3A occurs when the motor is prevented from

rotating, such as when encountering an obstacle. The motor driver and power supply must handle these brief current surges without damage.

The gear ratio of 1:48 to 1:120 determines the trade-off between speed and torque. Higher ratios provide more torque but lower speed. The optimal ratio depends on wheel diameter, robot weight, and desired performance characteristics.

Parameter	Specification
Type	12V or 5V DC motor with gearbox
RPM	100-300 RPM (varies by application)
Torque	Sufficient to move robot chassis and water container
Current (normal load)	0.5A to 1A
Stall Current	2A to 3A
Gear Ratio	1:48 to 1:120

3.4.2 Differential Drive System

The robot uses a differential drive system with two DC motors mounted on opposite sides of the chassis. This configuration is simple, efficient, and provides excellent manoeuvrability for indoor applications. By controlling the relative speeds and directions of the two motors, various movement patterns can be achieved.

When both motors rotate in the same direction at the same speed, the robot moves forward in a straight line. This is the basic movement mode for approaching fires detected directly ahead. Equal speeds ensure straight-line tracking without veering.

When both motors rotate in the opposite direction at the same speed, the robot moves backward. Reverse movement may be used for repositioning or retreating from dangerous situations. Care must be taken to maintain stability and avoid tipping.

To turn left, the left motor rotates slower or in reverse compared to the right motor. The differential speed causes the robot to arc toward the left. Sharp turns use reverse rotation on the inside motor, while gentle turns use simply reduced speed.

To turn right, the right motor rotates slower or in reverse compared to the left motor. This mirrors the left turn behaviour, enabling navigation in either direction.

When motors rotate in opposite directions at the same speed, the robot rotates in place around its centre. This zero-radius turning provides maximum manoeuvrability in confined spaces and enables rapid reorientation without moving forward or backward.

3.5 Water Pump System

The water pump system is responsible for extinguishing the fire once the robot has navigated to its location. The system consists of a small DC water pump, a water container, tubing, and a nozzle.

When activated, the pump draws water from the container and pushes it through the tubing, and the nozzle directs the water flow toward the fire with proper force. This system is controlled by the microcontroller and activates when fire is detected at close range.

The extinguishing system design balances effectiveness with practical constraints. The water capacity must be sufficient to suppress expected fires while keeping robot weight manageable.

The pump flow rate must deliver adequate water for suppression without excessive power consumption. The nozzle design must direct water effectively while tolerating the robot’s movement and positioning variations.

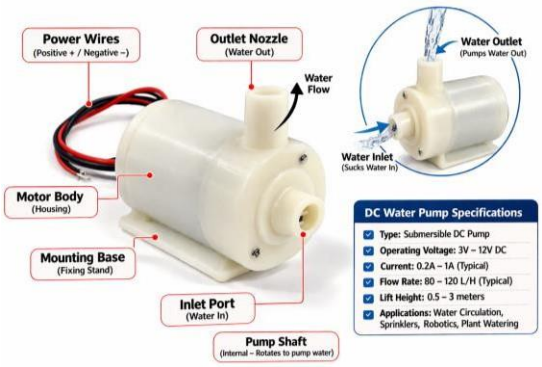


FIG 5. DC Water Pump

3.5.1 DC Water Pump

A small 12V or 5V DC water pump is used for the extinguishing mechanism. These pumps are commonly used in aquariums, water dispensers, and small fountain systems, making them readily available and affordable. They are compact, reliable, and can provide sufficient water pressure for fire extinguishing at close range.

The operating voltage of 12V or 5V matches common battery configurations and motor supplies. This compatibility simplifies power system design by allowing shared power sources. Current consumption of 0.5A to 1A is manageable for battery-powered operation.

The flow rate of 1-2 liters per minute provides adequate water delivery for small fire suppression. While professional firefighting equipment delivers much higher flow rates, the Fire Fighting Bot targets smaller fires where moderate water delivery is sufficient. The limited water capacity also makes high flow rates impractical.

The maximum head of 1-3 meters indicates the pump's ability to lift water vertically. This capability ensures adequate pressure for horizontal spraying at robot height. Higher head ratings provide more pressure but typically require more power.

The inlet and outlet diameter of 8-10 mm matches common tubing sizes, simplifying assembly and replacement. Standard sizes ensure availability of compatible tubing and fittings.

Parameter	Specification
Operating Voltage	12V or 5V DC
Current	0.5A to 1A
Flow Rate	1-2 liters per minute
Maximum Head	1-3 meters
Inlet/Outlet Diameter	8-10 mm

3.5.2 Relay Module

Since the microcontroller cannot directly provide sufficient current to operate the water pump, a relay module is used as an electronic switch. The relay is controlled by a digital output signal from the ESP32 microcontroller, allowing safe isolation between low-power and high-power circuits. When the ESP32 sends a signal, the relay gets activated and completes the circuit to power the pump. This enables the system to control the pump automatically when fire is detected.

A typical 5V relay module includes a control input pin connected to the ESP32, along with VCC (5V power supply) and GND (ground). It also has Normally Open (NO) and Common (COM) terminals used to connect the water pump circuit. When the relay is triggered, the NO contact closes with COM, allowing current to flow to the pump. This setup ensures efficient and safe operation of the fire extinguishing mechanism.

The relay provides electrical isolation between the sensitive microcontroller circuitry and the potentially noisy pump motor. This isolation protects the ESP32 from voltage spikes and electrical noise generated by the pump. The relay also allows the low-voltage control circuit to switch high-voltage, high-current loads safely.

The SRD-05VDC-SL-C relay module used in this project features a 5V coil that can be driven directly from ESP32 GPIO pins with appropriate current limiting. The relay contacts are rated for 10A at 250VAC or 10A at 30VDC, providing

ample margin for the water pump load. An LED indicator shows relay state for debugging and monitoring.

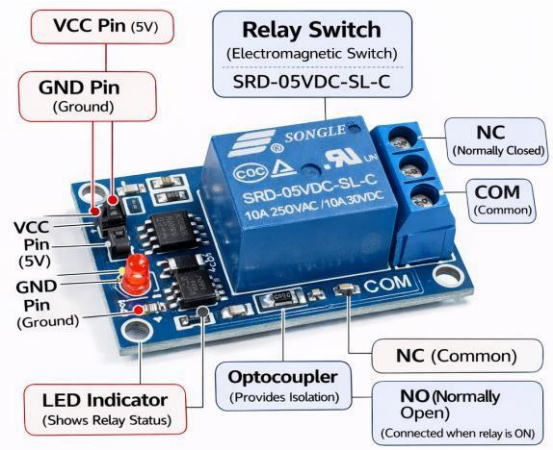


FIG 6. Relay Switch (SRD-05VDC-SL-C)

3.6 Servo Motor

The servo motor is used to position the water nozzle, allowing precise aiming of the water spray toward the fire source. This improves the efficiency of the extinguishing process by ensuring that water is directed where it will be most effective.

Without servo positioning, the water would spray in a fixed direction, potentially missing the fire or wasting water on non-burning areas.

Servo motors are ideal for this application because they provide accurate position control without requiring feedback sensors or complex control algorithms.

The position is set by a simple PWM signal, and the servo maintains that position against moderate loads. This simplicity reduces development time and improves reliability.

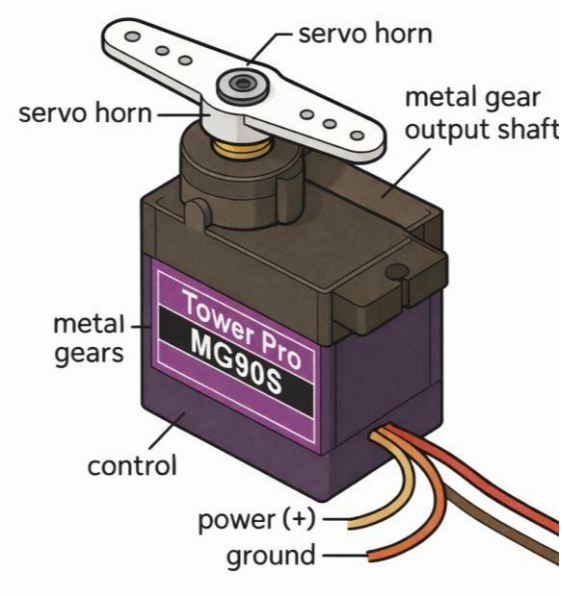


FIG 7. MG90S Servo Motor

Parameter	Specification
Type	Standard RC servo (SG90 or MG995)
Operating Voltage	4.8V – 6V
Torque	1.8-2.5 kg/cm (SG90) or 10-15 kg/cm (MG995)
Rotation Angle	0-180 degrees
Control Signal	PWM (1-2 ms pulse width)
Operating Speed	0.12 sec/60 degrees (at 4.8V)

The servo motor is controlled using PWM signals from the ESP32. By adjusting the pulse width between 1ms and 2ms, the servo can be positioned at any angle between 0 and 180 degrees. A 1.5ms pulse positions the servo at 90 degrees (center), 1ms pulse at 0 degrees, and 2ms pulse at 180 degrees. Intermediate pulse widths produce proportional positions.

The servo sweeps across its range while monitoring flame sensor readings to find the direction with the strongest fire detection. Once identified, the servo positions the nozzle at that angle for optimal water spray coverage. This active aiming

process compensates for robot positioning errors and ensures effective extinguishing.

The MG90S servo offers metal gears for Improved durability compared to plastic-gear alternatives. This durability is important for the repeated movements required during firefighting operations. The compact size fits within the robot chassis without excessive space requirements.

3.7 Buck Converter

The buck converter is a DC-DC step-down converter that efficiently converts a higher input voltage to a lower output voltage.

In the Fire Fighting Bot, it is used to obtain a stable 5V supply from the 12V battery for powering the ESP32, sensors, and servo motor. This voltage conversion is essential because different components require different operating voltages.

The buck converter operates by rapidly switching the Input voltage on and off, then filtering the result to produce a steady lower voltage.

The switching action minimizes power loss compared to linear regulators, which dissipate excess voltage as heat. This efficiency is particularly important for battery-powered applications where every milliampere-hour counts.

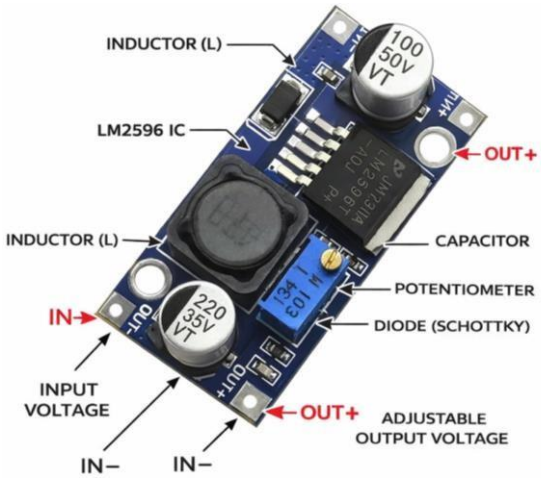


FIG 8. Buck Converter LM2596

Parameter	Specification
Input Voltage	4.5V - 28V
Output Voltage	0.8V - 20V (adjustable, set to 5V)
Output Current	Up to 3A
Efficiency	Up to 96%
Switching Frequency	1.5 MHz
Protection	Over-current, over-temperature, short-circuit

The buck converter provides several advantages over linear regulators like the LM7805. Higher efficiency (up to 96% versus approximately 50% for linear regulators) means less power wasted as heat and longer battery life. Less heat generation reduces thermal management requirements, simplifying enclosure design. Better battery life extends operating time between charges. Compact size fits within the robot chassis without excessive space requirements.

The LM2596 module used in this project provides adjustable output voltage set by a potentiometer. For the Fire Fighting Bot, the output is adjusted to 5V to power the ESP32 (through its onboard regulator), sensors, servo, and relay module. The 3A current capacity provides ample margin for the expected load.

Built-in protection features include over-current protection to limit output current during faults, over-temperature protection to prevent damage from excessive heat, and short-circuit protection to survive accidental output shorts. These protections improve reliability and prevent damage from wiring errors or component failures.

3.8 Power Supply

The Fire Fighting Bot requires a reliable power supply system to operate all components. Proper power management is critical for reliable operation and reasonable battery life. The power requirements include multiple voltage levels for different components, necessitating careful design and component selection.

The ESP32 requires 3.3V, supplied through its onboard regulator from the 5V buck converter output. While the ESP32 can accept 5V input through its voltage regulator, providing a clean 3.3V supply ensures stable operation and protects against voltage transients.

The DC motors require 12V directly from the battery for maximum torque and speed. Operating motors at lower voltages reduces performance and may cause overheating due to increased current draw. The direct battery connection minimizes voltage drops that would occur with long supply lines.

The water pump also requires 12V directly from the battery. Like the motors, the pump performs best at its rated voltage. The high current demand of the pump during startup requires robust wiring and good battery capacity.

The sensors and servo motor require 5V from the buck converter. This regulated voltage ensures consistent sensor readings and reliable servo operation despite battery voltage variations as the battery discharges.

The relay module requires 5V from the buck converter for its control circuitry. The relay coil current is within the ESP32 GPIO capability with appropriate current limiting.

A common approach is to use a 12V rechargeable battery pack (such as Li-ion or Li-Po) with the buck converter providing the necessary 5V for the ESP32, sensors, and servo. The motors and pump receive power directly from the 12V supply. This configuration provides efficient power distribution with minimal conversion losses.

Battery capacity should be selected based on expected operating time. A 2000mAh to 5000mAh battery pack typically provides sufficient runtime for testing and demonstration purposes. For the automatic mode with Wi-Fi connectivity, a larger capacity battery (5000mAh or more) is recommended to account for the additional power consumption of the ESP32's wireless transmission.

Battery selection involves trade-offs between capacity, weight, cost, and safety. Lithium-ion batteries offer high energy density but require careful charging and protection circuits. Lithium-polymer batteries provide similar performance in

flexible form factors. Lead-acid batteries are heavier but more tolerant of abuse. The choice depends on specific application requirements and constraints.

Power consumption budgeting helps ensure adequate battery capacity. Typical consumption includes: ESP32 active with Wi-Fi (100-200mA), motors during movement (500mA-1A each), pump during operation (500mA-1A), servo during positioning (100-200mA), and sensors and relay (minimal). Peak consumption during simultaneous motor and pump operation may reach 3-4A, requiring battery and wiring capable of delivering such currents.

4. SYSTEM DESIGN AND ARCHITECTURE

This chapter presents the overall system design of the Fire Fighting Bot, including the block diagram, circuit design, and mechanical structure. The design follows a modular approach to ensure ease of construction, debugging, and future modifications. Each subsystem is designed to function independently while integrating seamlessly with the overall system.

4.1 Block Diagram

The Fire Fighting Bot system can be represented by a block diagram showing the interconnection of various components. This high-level view helps understand how information and power flow through the system, and how the various subsystems interact to achieve the overall firefighting function.

The system consists of four main blocks: Input Block, Processing Block, Output Block, and Power Supply Block. Each block contains multiple components that work together to perform specific functions within the overall system.

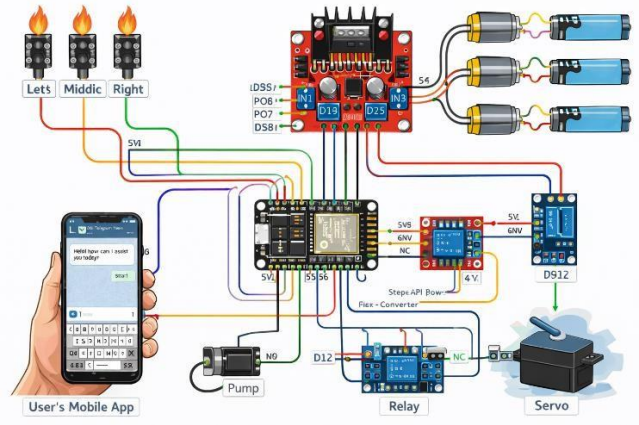


FIG 9. Circuit Diagram

4.1.1 Input Block

The input block consists of three flame sensors positioned at left, center, and right positions on the robot. These sensors continuously monitor the environment for infrared radiation emitted by fire. The analog output from each sensor is fed to the ESP32's analog input pins for processing. Additionally, in automatic mode, the

Telegram Bot API provides mission trigger commands as input, allowing remote operation.

The flame sensors provide the primary sensory input for fire detection. Their placement at different angles enables directional detection, allowing the robot to determine not just whether fire is present but also approximately where it is located. This directional information is essential for navigation toward the fire.

The Telegram input provides high-level commands that initiate autonomous operation. These commands come from human operators who may be monitoring the facility remotely. The integration of human oversight with autonomous execution combines the strengths of both approaches.

4.1.2 Processing Block

The ESP32 microcontroller on the MOTOR BOARD serves as the processing block. It reads the sensor values, processes them according to the programmed algorithm, and makes decisions about robot movement, servo positioning, and extinguishing actions. The processing involves comparing sensor readings against thresholds, determining the appropriate response, and controlling the various output devices.

The ESP32's dual-core architecture allows parallel processing of different tasks. One core can handle sensor reading and motor control while the other manages Wi-Fi communication and Telegram API interaction. This parallel processing ensures responsive operation even during network communication.

The processing algorithms implement a state machine that governs robot behavior. States include IDLE (waiting for commands), SEARCH (scanning for fire), APPROACH (moving toward detected fire), POSITION (aiming the nozzle), EXTINGUISH (activating the pump), and VERIFY (confirming fire suppression). Transitions between states depend on sensor inputs and mission progress.

4.1.3 Output Block

The output block includes the motor driver, DC motors, servo motor, and water pump system. Based on the processing results, the ESP32 sends control signals to

the motor driver to control robot movement, positions the servo for nozzle aiming, and activates the relay to turn the water pump on or off.

The motor driver receives control signals from the ESP32 and converts them to the power levels needed to drive the DC motors. PWM signals enable speed control, while digital signals control direction. The dual H-bridge configuration allows independent control of both motors for differential steering.

The servo motor receives PWM position commands from the ESP32. The servo moves to the commanded position and maintains it against moderate loads. This position control enables precise nozzle aiming without requiring continuous adjustment.

The water pump relay receives a digital control signal from the ESP32. When activated, the relay closes contacts that connect the pump to the battery, causing water to flow. When deactivated, the relay opens the circuit, stopping the pump.

4.1.4 Power Supply Block

The power supply block provides the necessary voltages to all components. A 12V battery pack serves as the main power source, with the buck converter providing 5V for the ESP32, sensors, and servo, while the motors and pump receive power directly from the 12V supply.

The power distribution must handle varying loads as different components activate. Motors and pumps draw significant current during operation, while sensors and control electronics have more modest requirements. Proper wire sizing and connection design ensure adequate power delivery without excessive voltage drops.

Battery monitoring can be implemented to track remaining capacity and predict operating time. While not included in the basic design, this capability would enhance operational planning and prevent unexpected shutdowns.

4.2 Circuit Design

The circuit design of the Fire Fighting Bot involves connecting all components to the BOARD according to a carefully planned scheme. Proper circuit design ensures reliable operation, minimizes interference between components, and simplifies troubleshooting.

4.2.1 Flame Sensor Connections

Three flame sensors are connected to the ESP32 analog pins for fire detection. Each sensor requires three connections: VCC (5V), GND (Ground), and OUT (Analog output to ESP32). The analog outputs provide continuous readings proportional to detected infrared radiation.

The Left Flame Sensor is connected to GPIO 34 (Analog Pin). This pin is one of the ESP32's ADC channels capable of 12-bit resolution. The sensor output voltage is converted to a digital value between 0 and 4095, with lower values typically indicating stronger fire detection.

The Center Flame Sensor is connected to GPIO 35 (Analog Pin). As the primary forward-facing sensor, this connection provides the most important fire detection information for navigation. Strong detection on this sensor indicates the robot is aligned with the fire.

The Right Flame Sensor is connected to GPIO 32 (Analog Pin). This completes the triad of sensors, enabling detection of fires on the right side. Together with the left sensor, it provides the information needed for directional navigation.

Each flame sensor requires proper power supply decoupling to prevent noise from affecting readings. A 100nF ceramic capacitor between VCC and GND near each sensor helps filter power supply noise. The analog connections should be kept short and away from high-current motor wiring to minimize interference.

4.2.2 Motor Driver Connections

The integrated motor driver connects to the ESP32 for control signal input. The motor driver receives both direction control signals and PWM speed control signals from the ESP32 GPIO pins.

ENA (Motor A Enable) is connected to GPIO 25 (PWM). This PWM signal controls the speed of Motor A (typically the left motor). PWM values from 0 to 255 produce speeds from stopped to full speed.

IN1 is connected to GPIO 26. This digital signal, along with IN2, determines Motor A direction. The combination of IN1 and IN2 states produces forward, reverse, or brake operation.

IN2 is connected to GPIO 27. Together with IN1, this signal controls Motor A direction. The logic level combinations produce the different direction states.

ENB (Motor B Enable) is connected to GPIO 33 (PWM). This PWM signal controls the speed of Motor B (typically the right motor), matching the function of ENA for Motor A.

IN3 is connected to GPIO 14. This digital signal controls Motor B direction, corresponding to IN1 for Motor A.

IN4 is connected to GPIO 12. This digital signal completes the Motor B direction control, corresponding to IN2 for Motor A.

The motor driver's power connections include +12V connected to the battery positive terminal and GND connected to the battery negative and ESP32 GND. Proper wire sizing is essential for these high-current connections to prevent voltage drops and overheating.

4.2.3 Servo Motor Connection

The servo motor is connected to the ESP32 for position control. Servos require three connections: power, ground, and signal.

The Signal Pin is connected to GPIO 13 (PWM capable). This pin outputs the PWM signal that commands servo position. The ESP32's LED PWM functionality provides accurate pulse generation for reliable servo control.

VCC is connected to 5V from the buck converter. The servo requires 4.8V to 6V for proper operation. The buck converter's regulated 5V output provides clean power for consistent servo performance.

GND is connected to common ground. The ground connection must be solid to prevent ground loops and ensure reliable signal reference.

Servo wiring should be kept away from motor power wiring to prevent electrical noise from affecting position accuracy. If necessary, a small capacitor across the servo power connections can help filter noise.

4.2.4 Water Pump Connections

The water pump is controlled through a relay module that switches the high-current pump load using a low-current control signal from the ESP32.

Relay Control is connected to GPIO 15. This digital output from the ESP32 activates the relay when set HIGH. A current-limiting resistor protects the ESP32 GPIO pin from excessive current draw.

Relay VCC is connected to 5V from the buck converter. This powers the relay module's control electronics.

Relay GND is connected to Ground. This completes the control circuit power supply.

Pump positive is connected to the battery through the relay contacts. When the relay activates, these contacts close, completing the circuit and powering the pump.

Pump negative is connected to the battery negative. This completes the pump power circuit.

The relay provides electrical isolation between the sensitive ESP32 circuitry and the potentially noisy pump motor. This isolation protects the ESP32 from voltage spikes and electrical interference.

4.2.5 Complete Pin Mapping

The complete pin mapping summarizes all ESP32 GPIO connections for reference during assembly, troubleshooting, and modification. Understanding this mapping is essential for any changes to the hardware configuration.

Component	ESP32 Pin	Type
Left Flame Sensor	GPIO 34	Analog Input
Center Flame Sensor	GPIO 35	Analog Input
Right Flame Sensor	GPIO 32	Analog Input
Motor A Input 1	GPIO 26	Digital Output
Motor A Input 2	GPIO 27	Digital Output
Motor A Enable	GPIO 25	PWM Output
Motor B Enable	GPIO 33	PWM Output
Motor B Input 1	GPIO 14	Digital Output
Motor B Input 2	GPIO 12	Digital Output
Servo Motor	GPIO 13	PWM Output
Water Pump Relay	GPIO 15	Digital Output
Bluetooth RX (Manual)	GPIO 16	UART RX
Bluetooth TX (Manual)	GPIO 17	UART TX

4.3 Mechanical Structure

The mechanical structure of the Fire Fighting Bot is designed to be robust, stable, and capable of carrying the necessary components. The design considerations include weight distribution, center of gravity, ease of assembly, and maintenance access. A well-designed mechanical structure ensures reliable operation and simplifies construction.

4.3.1 Chassis Design

The chassis is designed using SolidWorks CAD software and manufactured using 3D printing with PLA material. Computer-aided design enables precise dimensioning and fit verification before physical construction. The design features several key elements that contribute to overall functionality.

The two-layer structure separates components by function. The bottom layer houses motors, wheels, and battery - the heavy components that benefit from low mounting. The top layer contains the BOARD, sensors, and water container - components that require access and visibility. This separation simplifies assembly and maintenance while optimizing weight distribution.

Integrated motor mounting brackets with proper alignment features ensure that motors are positioned correctly without additional hardware. The brackets include features for securing motors and maintaining wheel alignment. Proper alignment is essential for straight-line tracking and accurate steering.

The water container holder with quick-release mechanism enables rapid refilling without tools. This feature is important for operational readiness, allowing quick turnaround between missions. The holder positions the container securely while permitting easy removal.

Sensor mounting brackets at optimal angles position the flame sensors for effective fire detection. The angles are chosen to provide overlapping coverage while maintaining directional discrimination. The brackets protect sensors from physical damage while allowing adjustment if needed.

Cable management channels for organized wiring keep cables secure and protected during robot movement. Proper cable management prevents wires from catching on obstacles or being damaged by moving parts. The channels also improve the robot's appearance and simplify troubleshooting.

The caster wheel mounting position at the front provides stability with minimal friction. The caster allows the robot to turn smoothly while supporting the front of the chassis. Proper positioning ensures stable three-point contact with the ground.

4.3.2 Motor Mounting

DC motors are mounted on opposite sides of the chassis using integrated brackets in the 3D printed design. The motors are positioned such that the wheels are aligned and make proper contact with the ground. Proper motor mounting is essential for reliable locomotion.

Wheel alignment affects straight-line tracking and turning accuracy. Misaligned wheels cause the robot to drift or turn unexpectedly. The integrated mounting brackets include alignment features that position motors correctly during assembly.

A caster wheel or ball caster is mounted at the front for stability. This third contact point prevents the robot from tipping forward while allowing free turning. The caster must be positioned to maintain proper ground contact across expected surface variations.

Motor wiring exits through strain relief features that prevent wire damage from vibration and movement. Secure wiring prevents intermittent connections that could cause erratic behaviour.

4.3.3 Sensor Mounting

Flame sensors are mounted on the front of the robot at different angles using dedicated brackets. Proper sensor positioning is critical for effective fire detection and navigation.

The Left Sensor is angled 30-45 degrees to the left, providing detection coverage for the left front quadrant. This angle balances detection range with directional precision.

The Centre Sensor points straight ahead, providing detection directly in front of the robot. This sensor is most important for final approach to the fire.

The Right Sensor is angled 30-45 degrees to the right, mirroring the left sensor. Together, the three sensors provide comprehensive forward coverage.

The sensors are positioned at a height that provides good fire detection while avoiding interference from ground reflections. The optimal height depends on expected fire locations and ground surface characteristics.

4.3.4 Water Container and Pump

A plastic container with a capacity of 500ml to 1 litre serves as the water reservoir. This capacity provides sufficient water for small fire suppression while keeping

weight manageable. Larger containers would extend operating time but increase weight and reduce mobility.

The container is positioned on the top layer of the chassis to maintain the robot's centre of gravity near the middle. High mounting of heavy components would make the robot unstable, while low mounting might interfere with ground clearance.

The water pump is placed inside or near the container with tubing running to a nozzle mounted on the servo mechanism at the front of the robot. Short tubing runs minimize flow resistance and pressure loss.

The nozzle design affects spray pattern and range. A focused stream provides longer reach but smaller coverage area, while a wider spray covers more area with less range. The nozzle selection depends on expected fire sizes and operating distances.

5. WORKING PRINCIPLE

The Fire Fighting Bot operates through a sequence of detection, navigation, and extinguishing processes. This chapter explains the working principles behind each phase of operation, providing the theoretical foundation for understanding and modifying the system behaviour.

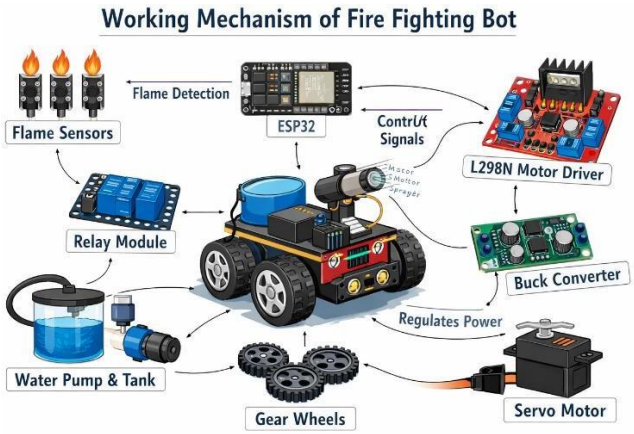


FIG 10. Working Mechanism

5.1 Fire Detection Mechanism

The fire detection mechanism is based on the infrared radiation sensing capability of flame sensors. The process involves multiple steps that transform raw sensor readings into actionable fire detection information. Understanding this process is essential for calibrating sensors and troubleshooting detection issues.

5.1.1 Sensor Reading

The ESP32 continuously reads analog values from all three flame sensors using the analogRead() function. The readings range from 0 to 4095 (12-bit ADC on ESP32), where lower values typically indicate stronger fire detection (depending on sensor configuration). Continuous sampling ensures timely detection without excessive processor load.

The sensor reading can be represented as a voltage calculation. For a 3.3V reference voltage, the conversion formula is: Sensor Value = (Analog Reading /

4095) x Reference Voltage. This gives a voltage reading between 0 and 3.3V that can be compared to expected sensor behavior.

The analog-to-digital conversion process involves sampling the analog voltage and quantizing it to a digital value. The ESP32's ADC uses successive approximation to achieve 12-bit resolution. Conversion time is typically a few microseconds per sample, allowing rapid multi-sensor reading.

Reading all three sensors in quick succession provides a snapshot of the fire detection environment. This snapshot is used to determine fire presence and direction. The reading frequency balances detection responsiveness with processor availability for other tasks.

5.1.2 Threshold Detection

A threshold value is set to determine when a fire is detected. When the sensor reading falls below this threshold (for sensors where lower values indicate stronger fire), the system recognizes the presence of fire. Threshold selection is critical for reliable operation.

The threshold value is determined through calibration and depends on factors such as sensor sensitivity, ambient lighting conditions, and expected fire distance. A typical threshold range is 2000-3000 for ESP32 analog readings (12-bit). Values in this range provide good detection while minimizing false positives.

Calibration involves testing sensors with known fire sources at various distances to establish appropriate thresholds. The calibration process should be repeated for different ambient lighting conditions, as background infrared radiation affects sensor readings.

Hysteresis can be implemented to prevent rapid toggling around the threshold. A higher threshold for detecting fire and a lower threshold for clearing detection creates a dead band that stabilizes detection state.

5.1.3 Direction Determination

By comparing the readings from all three sensors, the system determines the direction of the fire. This directional information guides navigation toward the fire source.

If the left sensor reading is below threshold and is the lowest among all sensors, fire is detected on the left. The robot should turn left to center the fire in its path.

If the center sensor reading is below threshold and is the lowest, fire is detected directly in front. The robot should move forward toward the fire.

If the right sensor reading is below threshold and is the lowest, fire is detected on the right. The robot should turn right to align with the fire.

If multiple sensors detect fire, the fire is in the direction of the lowest reading. This situation may indicate a large fire or that the robot is close to the fire source.

5.2 Navigation Algorithm

The navigation algorithm controls the robot's movement based on fire detection results. The algorithm follows a simple but effective approach that balances responsiveness with stability.

5.2.1 Basic Navigation Logic

The navigation logic can be summarized as a set of rules that determine robot movement based on fire detection state. These rules implement a fire-following behavior that guides the robot toward detected fires.

If no fire is detected, the robot rotates in place to search for fire. This scanning behavior ensures that the robot eventually detects fires in any direction. The rotation speed balances search coverage with detection sensitivity.

If fire is detected on the left, the robot turns left while moving forward. This combined movement both approaches the fire and centers it in the robot's path. The turn rate depends on how far left the fire is detected.

If fire is detected on the right, the robot turns right while moving forward. This mirrors the left detection behavior for fires on the right side.

If fire is detected in front, the robot moves straight forward. This is the simplest case, requiring only forward movement without turning.

If fire is very close (all sensors active), the robot stops and activates the pump. This condition indicates that the robot has reached the fire and should begin extinguishing.

5.2.2 Motor Control

Motor control is achieved through the integrated motor driver using specific input signal patterns. By varying the HIGH and LOW signals sent from the microcontroller, the motors can be controlled to move forward, backward, or stop. This method allows precise direction control and smooth navigation of the Fire Fighting Bot.

Movement	IN1	IN2	IN3	IN4	ENA	ENB
Forward	HIGH	LOW	HIGH	LOW	PWM	PWM
Backward	LOW	HIGH	LOW	HIGH	PWM	PWM
Turn Left	LOW	HIGH	HIGH	LOW	PWM	PWM
Turn Right	HIGH	LOW	LOW	HIGH	PWM	PWM
Stop	LOW	LOW	LOW	LOW	0	0

5.2.3 Speed Control

Motor speed is controlled using PWM (Pulse Width Modulation) signals on the ENA and ENB pins. PWM values range from 0 (stopped) to 255 (full speed). The speed can be adjusted based on the fire detection intensity to optimize approach behavior.

Normal speed of 180-220 is used for general movement when fire is detected at moderate distance. This speed provides good progress while maintaining control and allowing time for sensor updates.

Slow speed of 120-180 is used when approaching fire closely. Reduced speed provides finer control for positioning and reduces overshoot. The slower approach also allows more time for sensor updates as the fire gets closer.

Search speed of 200-255 is used when searching for fire. Higher speed covers more area quickly, reducing the time to find fires. The maximum speed is limited by the need to detect fires before passing them.

5.3 Extinguishing Process

The extinguishing process is triggered when the robot determines that it is close enough to the fire. The process involves multiple steps that ensure effective fire suppression while managing water consumption and system safety.

5.3.1 Proximity Detection

Proximity to the fire is determined by the intensity of sensor readings. When all three sensors detect fire with readings below a critical threshold, the system assumes the robot is close enough to extinguish the fire. This multi-sensor detection indicates that the fire is both close and large enough to warrant immediate action.

The critical threshold is set lower than the detection threshold to ensure the robot is very close before activating the pump. This prevents wasting water on fires that are still too distant for effective suppression.

Proximity detection may be augmented with timing considerations. Requiring multiple consecutive readings below threshold prevents premature activation from brief detection spikes.

5.3.2 Nozzle Positioning

Before activating the pump, the servo motor positions the water nozzle toward the fire source. The servo sweeps through its range (0-180 degrees) while monitoring sensor readings to find the direction with the strongest fire detection. The nozzle is then positioned at that angle for optimal water spray coverage.

The sweep-and-find process maximizes the chance of hitting the fire even if the robot is not perfectly aligned. By actively seeking the strongest fire signal, the system compensates for positioning errors and fire movement.

The positioning speed balances thoroughness with responsiveness. A slower sweep provides more samples for better direction estimation, while a faster sweep reduces delay before extinguishing begins.

5.3.3 Pump Activation

When the proximity condition is met and the nozzle is positioned, the extinguishing sequence begins. The robot stops moving to ensure stable positioning during water discharge.

The servo positions the nozzle toward the fire at the angle determined during the sweep process. This positioning ensures water is directed where it will be most effective.

The ESP32 activates the relay by setting the control pin HIGH. This energizes the relay coil, closing the contacts and connecting the pump to power.

The relay closes the circuit, powering the water pump. Water flows from the container through the tubing and out the nozzle toward the fire.

Water is sprayed toward the fire through the nozzle for a preset duration, typically 5-10 seconds. This duration provides sufficient water for small fire suppression while conserving the limited water supply.

After the preset duration, the pump is turned off by setting the relay control pin LOW. The relay opens, disconnecting power from the pump.

The robot resumes searching for any remaining fire. If fire is still detected, the extinguishing process repeats. If no fire is detected, the mission is complete.

5.3.4 Safety Considerations

The extinguishing process includes safety features that protect the system and optimize performance.

Maximum pump runtime prevents water wastage and motor damage. If the pump runs longer than the preset duration, it is automatically shut off. This timeout protects against software errors or stuck relays that could drain the water supply or overheat the pump.

Cooldown period between pump activations allows the pump motor to dissipate heat. Continuous operation without cooling could damage the motor or reduce its lifespan.

Verification that fire is still present before activating pump prevents wasting water on false detections or already-extinguished fires. The system confirms fire presence immediately before each activation.

6. SOFTWARE IMPLEMENTATION

The software implementation of the Fire Fighting Bot involves programming the ESP32 microcontroller to coordinate sensor reading, motor control, servo positioning, and pump activation. This chapter covers the development environment, programming logic, and code structure without presenting actual code listings as requested.

6.1 Arduino IDE

The Arduino Integrated Development Environment (IDE) is used for writing, compiling, and uploading the program to the ESP32 board. The Arduino IDE provides a user-friendly interface that simplifies the development process while providing access to advanced features when needed.

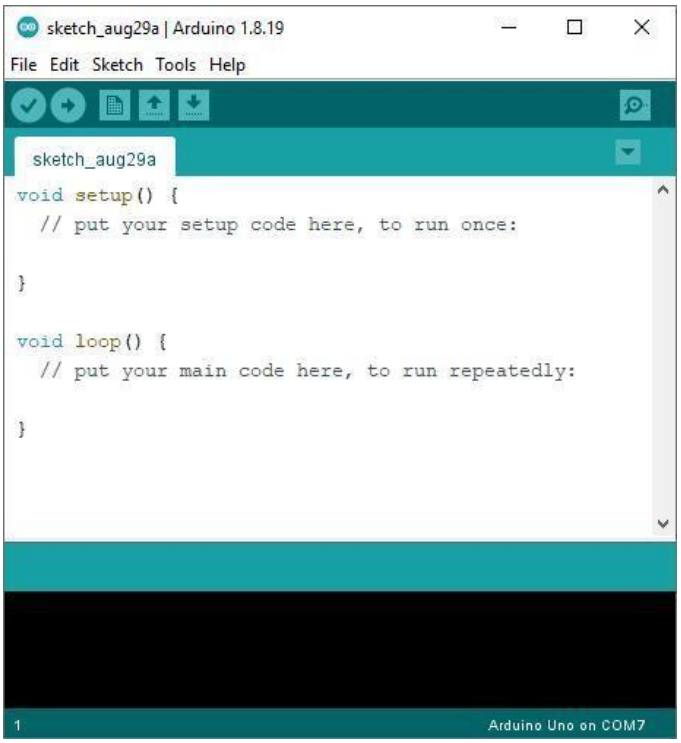


FIG 11. Arduino IDE Interface

6.1.1 IDE Features

The Arduino IDE provides a comprehensive set of features that support the entire development workflow from code creation to deployment.

The text editor with syntax highlighting for C/C++ code helps identify language elements and potential errors. Keywords, comments, strings, and other code elements are displayed in different colors for easy recognition. The editor also provides basic auto-indentation and bracket matching.

One-click compilation and upload to ESP32 boards simplifies the deployment process. The IDE handles the complex toolchain operations behind a simple button interface, reducing the learning curve for new developers.

The serial monitor provides debugging and data visualization capabilities. Program output can be displayed in real-time, and input can be sent to the running program. This feature is invaluable for troubleshooting and verifying program behavior.

Built-in examples and library manager provide starting points for common tasks and access to community-contributed code. The extensive library ecosystem covers sensors, communication protocols, and hardware interfaces.

Support for multiple board types including ESP32 enables development for various hardware platforms from a single environment. Board-specific configurations are handled automatically.

The integrated board manager for ESP32 core installation simplifies setup. Additional board support can be installed through a graphical interface without manual configuration.

6.1.2 ESP32 Board Setup

To program the ESP32 using Arduino IDE, the following setup is required. These steps ensure the development environment is properly configured for ESP32 development.

First, install the Arduino IDE from the official Arduino website. The IDE is available for Windows, macOS, and Linux platforms. Download the appropriate version for your operating system and follow the installation instructions.

Next, add ESP32 board support by opening File > Preferences and adding the ESP32 board manager URL to the Additional Board Manager URLs field. The URL provides access to the ESP32 core package.

https://dl.espressif.com/dl/package_esp32_index.json. This URL points to the official Espressif repository containing the ESP32 Arduino core.

Install the ESP32 board package by opening Tools > Board > Board Manager, searching for "ESP32", and clicking Install. This downloads and installs all necessary files for ESP32 development.

Select the appropriate board by opening Tools > Board > ESP32 Arduino > ESP32 Dev Module. This configures the IDE for the specific ESP32 module used in the project.

Install required libraries including WiFi for wireless connectivity, HTTPClient for Telegram API communication, and ServoESP32 for servo motor control. Libraries can be installed through the Library Manager or by manual download.

6.2 Programming Logic

The programming logic for the Fire Fighting Bot follows a structured approach with clear states and transitions. This organization ensures reliable operation and simplifies debugging and modification.

6.2.1 State Machine

The robot operates as a state machine with well-defined states that correspond to different phases of firefighting operation. Each state has specific entry conditions, behaviors, and exit transitions.

The IDLE state represents waiting for Telegram mission command in automatic mode. In this state, the robot is stationary and monitoring the Telegram API for incoming commands. The Wi-Fi connection is maintained, and status may be

reported periodically. Entry to this state occurs after system initialization and after mission completion. Exit from this state occurs when a START MISSION command is received.

The SEARCH state represents the robot rotating to find fire. In this state, the robot rotates in place while continuously reading flame sensors. Entry occurs upon mission start or after extinguishing when fire is no longer detected. Exit occurs when fire is detected by any sensor.

The APPROACH state represents the robot moving toward detected fire. In this state, the robot moves forward while adjusting direction based on sensor readings to center the fire. Entry occurs when fire is detected during search. Exit occurs when the fire is very close (all sensors active) or if fire is lost.

The POSITION state represents the servo positioning nozzle toward fire. In this state, the servo sweeps across its range to find the direction with strongest fire detection. Entry occurs when approaching completes. Exit occurs when optimal position is found.

The EXTINGUISH state represents the robot activating water pump. In this state, the pump runs for a preset duration while the servo maintains position. Entry occurs after positioning completes. Exit occurs when pump timeout expires.

The VERIFY state represents the robot checking if fire is extinguished. In this state, sensors are read to determine if fire is still present. Entry occurs after extinguishing completes. Exit occurs to EXTINGUISH if fire persists or to COMPLETE if fire is out.

The COMPLETE state represents mission completed, returning to IDLE. In this state, a completion message is sent via Telegram and the system prepares for the next mission. Entry occurs when fire is verified as extinguished. Exit occurs to IDLE after cleanup.

6.2.2 Algorithm Flow

The main algorithm follows a structured flow that coordinates all system functions. This flow ensures systematic operation while handling unexpected conditions gracefully.

Initialization is the first step, setting up all pins, Wi-Fi connection, and serial communication. This includes configuring GPIO modes, establishing Wi-Fi connectivity, initializing the servo position, and setting initial variable values. Proper initialization is essential for reliable operation.

After initialization, the system waits for Telegram mission command in automatic mode. This waiting state conserves power while maintaining connectivity. The system periodically checks for new messages from the Telegram Bot API.

When a mission command is received, the system enters the main operational loop. The loop begins by reading all flame sensors to assess the current fire detection state. These readings drive subsequent decisions.

The system checks if any sensor detects fire by comparing readings against the detection threshold. This check determines whether fire is present and guides the subsequent navigation decisions.

If no fire is detected, the robot rotates to search for fire and the loop returns to sensor reading. This search behavior ensures continuous scanning until fire is found.

If fire is detected, the system determines direction from sensor readings by comparing the relative strengths from left, center, and right sensors. The direction with the strongest detection indicates the fire's location.

The robot moves in the direction of fire, turning and moving forward as appropriate to center the fire in its path. Movement continues while fire remains detected.

The system checks if fire is close by comparing all sensor readings against the critical threshold. When all sensors show strong detection, the fire is assumed to be at close range.

If fire is close, the robot stops, positions the servo, and activates the pump for the preset duration. This extinguishing sequence suppresses the fire with directed water spray.

After extinguishing, the system returns to sensor reading to verify the fire is completely out. If fire persists, the extinguishing process repeats. If no fire is detected after extinguishing, the mission is complete.

6.3 Code Explanation

The software implementation consists of multiple functional components that work together to achieve autonomous firefighting. This section explains the purpose and operation of each major component without presenting actual code.

The library includes section imports necessary functionality for Wi-Fi connectivity, HTTP communication, servo control, and ESP32-specific features. These libraries provide pre-built functions that simplify development and improve reliability.

The configuration section defines constants and parameters that control system behavior. This includes pin assignments, threshold values, timing parameters, and Telegram API credentials. Centralizing these values simplifies modification and ensures consistency.

Global variables maintain state information across function calls. These include current state, sensor readings, servo position, and mission status. Proper variable scope ensures data is available where needed while preventing unintended modifications.

The setup function runs once at system startup, performing initialization tasks. This includes configuring GPIO pins, initializing serial communication, connecting to Wi-Fi, setting up the servo, and initializing variables. Successful setup is essential for proper operation.

The loop function runs repeatedly after setup, implementing the main operational logic. Each iteration reads sensors, updates state, and performs appropriate actions. The loop structure ensures continuous operation without blocking.

Sensor reading functions acquire data from the flame sensors and convert it to usable values. These functions handle the analog-to-digital conversion and may apply filtering or averaging to improve reliability.

Motor control functions implement movement commands by setting appropriate GPIO states and PWM values. These functions provide a high-level interface that hides the details of motor driver operation.

Servo control functions position the nozzle by generating appropriate PWM signals. These functions handle the conversion from desired angle to pulse width and manage servo movement timing.

Pump control functions activate and deactivate the water pump through the relay module. These functions include safety timeouts to prevent excessive operation.

Telegram communication functions handle message sending and receiving through the Bot API. These functions format HTTP requests, parse responses, and extract command information.

State machine functions implement transitions between operational states. These functions determine the appropriate next state based on current conditions and execute entry and exit actions.

7. MANUAL MODE - BLUETOOTH CONTROL

The Manual Mode of the Fire Fighting Bot allows operators to control the robot remotely using a Bluetooth RC app on a smartphone. This mode is useful for navigating complex environments where autonomous operation may be challenging, and provides direct human control for precise positioning and operation.

7.1 Bluetooth Module HC-05

The HC-05 Bluetooth module provides wireless serial communication between the robot and the smartphone. It is configured to work in slave mode, receiving commands from the RC app. The module's simplicity and reliability make it a popular choice for Bluetooth-enabled projects.

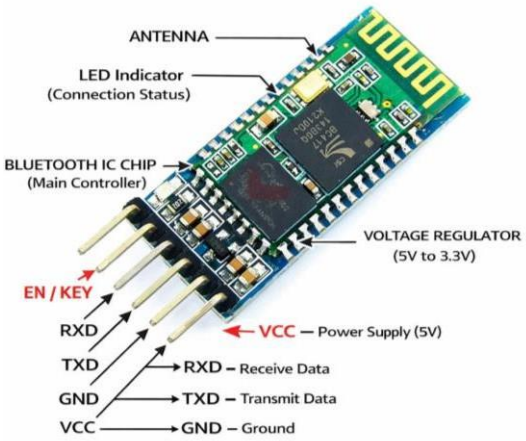


FIG 12. HC-05 Bluetooth Module

Parameter	Specification
Bluetooth Version	v2.0+EDR
Operating Frequency	2.4 GHz ISM band
Operating Voltage	3.3V - 6V (typically 5V)
Communication	UART (TTL)
Default Baud Rate	9600 bps

Parameter	Specification
Range	Up to 10 meters
Operating Temperature	-20°C to +55°C

The HC-05 module connects to the Arduino Uno (for manual mode) or ESP32 (for dual-mode operation) via UART serial communication. The connection uses standard serial pins for data transfer.

VCC is connected to 5V to power the module. The HC-05 can operate from 3.3V to 6V, with 5V being the most common configuration when interfacing with 5V microcontrollers.

GND is connected to Ground, providing the reference voltage for all signals. A common ground connection is essential for reliable communication.

TXD is connected to the RX pin of the microcontroller. This connection carries data from the HC-05 to the microcontroller.

RXD is connected to the TX pin of the microcontroller through a voltage divider for 3.3V. The voltage divider is necessary because the HC-05's RX pin is not 5V tolerant, while many microcontrollers output 5V signals.

The HC-05 module can be configured using AT commands sent through the serial interface. Configuration options include device name, pairing password, baud rate, and operating mode. Default settings work for most applications, but customization enables multiple robots to operate in the same area without interference.

7.2 RC Controller App

Various Bluetooth RC controller apps are available for Android and iOS devices. These apps provide user-friendly interfaces for controlling robots via Bluetooth communication. Popular options include Arduino Bluetooth Controller, Bluetooth RC Controller, and RoboRemo.

These apps typically provide several common features that simplify robot control. The directional pad or joystick interface enables intuitive movement control. Users can press directional buttons or drag a joystick to command robot movement.

Configurable buttons for additional functions allow customization of the control interface. Users can assign specific commands to buttons for functions like pump activation or mode switching.

Customizable command mapping enables adaptation to different robot configurations. The same app can control different robots by changing which commands are sent for each button press.

Real-time connection status display shows whether the Bluetooth connection is active and reliable. This feedback helps diagnose connection issues and confirms command transmission.

The app sends single-character commands via Bluetooth when buttons are pressed. Each character corresponds to a specific action that the robot interprets and executes. The simple command protocol minimizes latency and simplifies parsing.

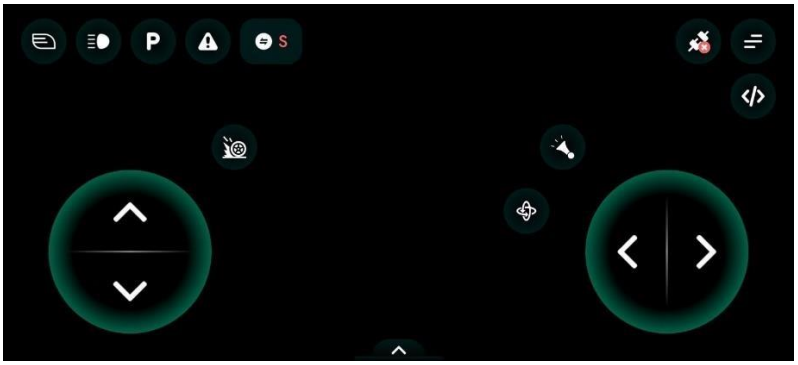


FIG 13. Controller

Command	Action
F	Move Forward
B	Move Backward

Command	Action
L	Turn Left
R	Turn Right
S	Stop
W	Activate Water Pump
w	Deactivate Water Pump

The command mapping uses single uppercase and lowercase letters for different functions. Uppercase letters typically control movement, while lowercase letters control secondary functions. This convention provides intuitive operation while maximizing the available command space.

7.3 Manual Control Implementation

The manual mode implementation provides responsive, intuitive control for operators. The key aspects of the implementation include serial communication setup, command parsing, motor control, and pump control.

Serial communication setup for the Bluetooth module configures the UART interface at 9600 baud. This baud rate provides reliable communication with the HC-05 module's default configuration. The serial buffer receives incoming commands for processing.

Command parsing in the main loop interprets incoming characters from the Bluetooth module. Each received character is compared against known command codes to determine the appropriate action. Invalid commands are ignored to prevent unintended behavior.

Motor control functions for forward, backward, left, right, and stop movements implement the corresponding robot behaviors. These functions set motor driver inputs appropriately for each command. PWM values control speed for smooth operation.

Pump control function activates or deactivates the water pump based on received commands. Safety features prevent continuous operation that could damage the pump or waste water.

Speed control using PWM enables smooth motor operation and variable speed. Higher PWM values produce faster movement for open areas, while lower values provide finer control for precise positioning.

The manual mode implementation prioritizes responsiveness to provide immediate feedback to operator commands. Minimal processing delay ensures that the robot reacts quickly to control inputs, improving the operator's ability to navigate effectively.

8. AUTOMATIC MODE - TELEGRAM INTEGRATION

The Automatic Mode of the Fire Fighting Bot enables autonomous operation triggered by Telegram messages. This innovative feature allows operators to initiate firefighting missions remotely, after which the robot autonomously detects fire, navigates toward it, and extinguishes it. The integration of cloud-based messaging with embedded systems demonstrates the potential of IoT for emergency response applications.

8.1 ESP32 WIFI Capabilities

The ESP32's integrated WIFI module provides the connectivity required for Telegram integration. This built-in capability eliminates the need for external communication modules, reducing cost and complexity while improving reliability.

The ESP32 supports 802.11 b/g/n protocols in the 2.4 GHz band, providing compatibility with virtually all consumer Wi-Fi networks. This broad compatibility ensures the Fire Fighting Bot can connect in most environments without special infrastructure.

Station mode enables the ESP32 to connect to existing Wi-Fi networks. In this mode, the robot joins a network just like any other device, obtaining an IP address through DHCP and communicating through the network's internet connection.

SoftAP mode allows the ESP32 to create its own access point. While not used in the current implementation, this capability could enable direct connection to the robot without requiring existing network infrastructure.

Mixed mode enables simultaneous Station and SoftAP operation. This advanced capability could support scenarios where the robot connects to the internet through one network while providing local access through its own access point.

Support for WPA/WPA2 security protocols ensures secure network connections. These protocols protect communication from eavesdropping and unauthorized access, essential for any internet-connected device.

Low power consumption modes extend battery life during standby operation. The ESP32 can reduce power consumption when Wi-Fi is not actively needed, waking periodically to check for messages.

The Wi-Fi connection is established in the setup function and maintained throughout operation. The ESP32 automatically reconnects if the connection is temporarily lost, ensuring robust communication. Connection status is monitored and reported through Telegram messages.

8.2 Telegram Bot API

The project uses Telegram Bot API to send and receive messages. Telegram provides a simple and efficient HTTP-based interface for communication between the ESP32 and the bot. The Bot API enables programmatic interaction with Telegram's messaging infrastructure.

The ESP32 interacts with the Telegram server using HTTP requests to receive commands and send status updates. These requests follow standard HTTP protocols, making them compatible with the ESP32's networking capabilities.

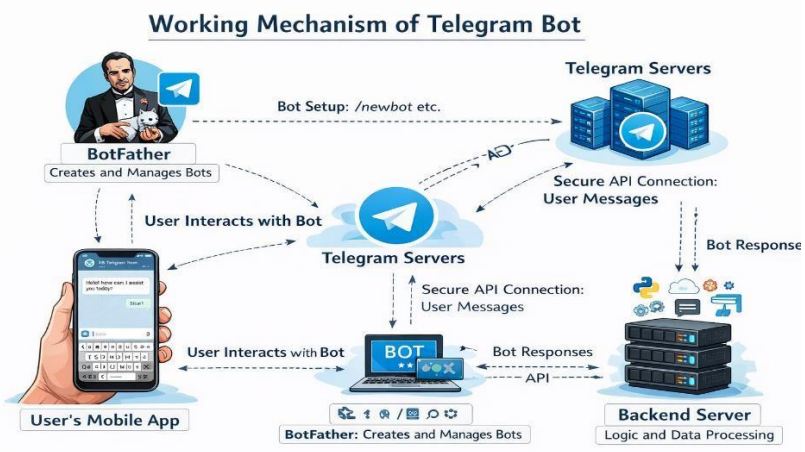


FIG 12. Working of Telegram API

8.2.1 API Setup

The setup process involves several steps to create and configure a Telegram bot for the Fire Fighting Bot project.

First, create a Bot using BotFather on Telegram. BotFather is Telegram's official bot for creating and managing other bots. Send the /newbot command and follow the prompts to create a new bot.

After creating the bot, obtain the Bot token provided by BotFather. This token is a unique identifier that authenticates API requests. The token must be kept secure, as anyone with the token can control the bot.

Get your Chat ID using the Telegram API or by sending a message to your bot and checking the update. The Chat ID identifies the conversation between you and the bot, ensuring messages are sent to the correct recipient.

Use the Bot Token and Chat ID in the ESP32 code to authenticate API requests and specify message recipients. These credentials are stored in the code and used for all Telegram communication.

8.2.2 HTTP Communication

The ESP32 uses the HTTPClient library to communicate with the Telegram Bot API using HTTPS requests. This library simplifies HTTP operations while providing full control over request parameters.

GET requests retrieve updates from the Telegram server, including new messages sent to the bot. The ESP32 polls periodically for updates, checking if new commands have been received.

POST requests send messages from the bot to the user. The ESP32 sends status updates, mission progress, and completion notifications through POST requests.

HTTPS encryption protects communication from interception. The ESP32 handles SSL/TLS negotiation to establish secure connections with Telegram servers.

JSON parsing extracts relevant information from API responses. The ESP32 parses the JSON format used by Telegram to extract message text, sender information, and other data.

8.3 Mission Flow

The automatic mode mission follows a structured flow from receiving the command to completing the firefighting task. This flow ensures systematic operation while providing status updates to the operator.

8.3.1 Mission Trigger

The mission is initiated when the ESP32 receives a specific command message via Telegram. Valid commands include several options for controlling robot operation.

The START MISSION command begins fire detection and extinguishing mission. Upon receiving this command, the robot transitions from IDLE to SEARCH state and begins autonomous operation.

The STATUS command requests current robot status. The robot responds with information about its current state, sensor readings, and mission progress.

The STOP command aborts the current mission and returns the robot to IDLE state. This command provides a safety mechanism to halt autonomous operation if needed.

The PUMP ON/OFF commands provide manual pump control. These commands allow direct pump operation independent of the autonomous sequence for testing or special situations.

8.3.2 Mission Execution

Once a START MISSION command is received, the robot executes a predefined sequence of operations designed to detect and extinguish fires autonomously.

First, the robot sends a confirmation message: "Mission started. Searching for fire..." This message confirms command receipt and indicates that autonomous operation has begun.

The robot enters SEARCH state, rotating to scan for fire. During this phase, all flame sensors are monitored continuously to detect any fire presence.

When fire is detected, the robot sends a message: "Fire detected at [direction]. Approaching..." The direction information helps operators understand the situation even when not physically present.

The robot enters APPROACH state, navigating toward the detected fire. Movement commands are generated based on sensor readings to center the fire in the robot's path.

When close to fire, the robot sends: "Positioning for extinguishing..." This message indicates that the robot is preparing to activate the extinguishing system.

The robot enters POSITION state, with the servo sweeping to find the optimal angle for water delivery. The angle with strongest fire detection is selected for nozzle positioning.

The robot enters EXTINGUISH state, activating the pump for the preset duration. Water is sprayed toward the fire at the positioned angle.

After extinguishing, the robot sends: "Fire extinguished. Verifying..." This message indicates that the primary extinguishing cycle has completed.

The robot enters VERIFY state, checking if fire is completely out. Sensors are read to determine if additional extinguishing is needed.

If fire persists, the process repeats from the EXTINGUISH state. Multiple cycles may be needed for complete suppression.

If fire is verified as out, the robot sends: "Mission completed successfully!" This message indicates successful completion of the firefighting mission.

The robot enters COMPLETE state and returns to IDLE, ready for the next mission.

8.3.3 Status Updates

Throughout the mission, the robot sends periodic status updates via Telegram. These updates keep operators informed of progress and enable monitoring of autonomous operation.

Current state information indicates what the robot is currently doing. States include SEARCHING, APPROACHING, EXTINGUISHING, and VERIFYING.

Sensor readings provide fire intensity and direction information. These readings help operators understand the fire situation and verify proper robot operation.

Battery level monitoring, if implemented, indicates remaining operating time. Low battery warnings enable proactive mission planning.

Error reporting communicates any problems encountered during operation. Errors might include sensor failures, communication problems, or mechanical issues.

8.4 Automatic Mode Implementation

The automatic mode implementation for ESP32 is more complex than the manual mode, incorporating Wi-Fi connectivity, HTTP communication, Telegram API integration, and the autonomous state machine. Key components of the implementation include several functional areas.

Wi-Fi connection management handles network connection establishment and maintenance. This includes scanning for networks, authenticating, obtaining IP addresses, and reconnecting if the connection is lost.

HTTP client functionality enables communication with the Telegram Bot API. The client handles request formatting, response parsing, and error handling for reliable message exchange.

Command parser interprets Telegram messages to extract commands and parameters. The parser recognizes valid commands and ignores or reports invalid messages.

State machine implementation governs mission flow through defined states and transitions. The state machine ensures systematic operation while handling unexpected conditions gracefully.

Sensor reading and fire detection algorithms process analog sensor values to determine fire presence and direction. These algorithms implement the threshold detection and directional comparison logic.

Motor control with PWM speed adjustment provides smooth, controllable movement. Speed is adjusted based on fire proximity to optimize approach behavior.

Servo control for nozzle positioning enables accurate water delivery. The servo sweeps to find optimal angles and maintains position during extinguishing.

Pump control with safety timeouts prevents excessive operation. Timeouts protect the pump and conserve water while ensuring adequate extinguishing duration.

Status reporting via Telegram messages keeps operators informed. Messages are sent at key mission milestones and when significant events occur.

The code structure follows best practices for ESP32 development. Non-blocking delays using millis() enable multitasking without stopping other operations. Watchdog timer prevents system hangs from infinite loops.

9. 3D PRINTED CHASSIS

The mechanical and electronic hardware integration is achieved 3D printed chassis. This chapter describes the design process, manufacturing methods, and assembly procedures for these critical components.

9.1 SolidWorks Chassis Design

The chassis was designed using SolidWorks, a professional 3D CAD software, to ensure precise dimensions and proper fit for all components. Computer-aided design enables virtual assembly and fit checking before physical manufacturing, reducing errors and iteration time.

9.1.1 Design Considerations

The chassis design considered multiple factors that affect performance, manufacturability, and usability. These considerations guided design decisions throughout the development process.

Weight distribution for stable movement was a primary consideration. Heavy components like the battery and motors are positioned low and centered to minimize the risk of tipping. The water container is positioned to maintain balance as water is consumed.

Component placement for easy access and maintenance ensures that parts can be reached for replacement, debugging, or modification. The BOARD is positioned for visibility of indicators and access to connectors.

Water container capacity and positioning balances the need for adequate extinguishing capability with weight and space constraints. The container must be large enough for effective operation but not so large as to impair mobility.

Sensor mounting angles for optimal fire detection were carefully selected. The angles provide overlapping coverage while maintaining directional discrimination. Mounting positions protect sensors from damage while allowing adjustment.

Motor mounting with proper wheel alignment ensures straight-line tracking and accurate steering. The design includes features that position motors correctly without requiring extensive adjustment.

Cable management for organized wiring keeps cables secure and protected. Channels and clips route wires away from moving parts and potential damage sources.

Aesthetic appearance for demonstration purposes was considered to create a professional-looking robot. Clean lines, organized wiring, and thoughtful detailing improve the visual impression.

9.1.2 Design Features

The final chassis design incorporates several features that address the design considerations and enable effective firefighting operation.

The two-deck structure separates components by function. The lower deck houses motors, battery, and wheels - the heavy, rugged components. The upper deck contains the BOARD, sensors, and water container - the sensitive, accessible components. This separation simplifies assembly and maintenance while optimizing weight distribution.

Integrated motor mounts with proper spacing ensure wheel alignment without additional hardware. The mounts include features for securing motors and maintaining position during operation.

Sensor mounting brackets at 30-degree angles position flame sensors for effective detection. The angles provide overlapping coverage while maintaining directional discrimination.

The water container holder with retention clips enables rapid refilling without tools. The clips hold the container securely while allowing quick removal for refilling.

The caster wheel mounting position at the front provides stability with minimal friction. The three-point contact with the ground ensures stability while allowing smooth turning.

Cable routing channels between decks keep wiring organized and protected. The channels guide cables away from moving parts and potential damage sources.

Ventilation holes for component cooling allow air circulation to prevent overheating. The holes are positioned to promote natural convection cooling of heat-generating components.

9.2 3D Printing in PLA

The chassis was manufactured using Fused Deposition Modeling (FDM) 3D printing with Polylactic Acid (PLA) filament. This manufacturing method provides design flexibility, rapid iteration, and low cost for prototype production.

9.2.1 Why PLA?

PLA was chosen for several reasons that make it well-suited for this application.

PLA is easy to print with minimal warping compared to other materials. This ease of use reduces print failures and improves success rates, especially for larger parts like the chassis.

Good dimensional accuracy ensures that printed parts match the CAD design closely. This accuracy is important for proper component fit and alignment.

Sufficient strength for the robot's weight and loads makes PLA adequate for this application. While not as strong as some engineering materials, PLA provides enough strength for the expected loads.

Environmental friendliness and biodegradability make PLA an environmentally responsible choice. Derived from renewable resources, PLA has lower environmental impact than petroleum-based plastics.

Wide color availability allows aesthetic choices for the robot appearance. Different colors can be used for different parts or for visual distinction.

Low cost compared to other filaments makes PLA economical for prototyping. Failed prints or design iterations are less costly with affordable material.

No heated chamber required for printing simplifies printer requirements. PLA prints well on basic printers without expensive upgrades.

9.2.2 Printing Parameters

Optimal printing parameters were selected to produce strong, accurate parts. These parameters balance print quality with print time and material usage.

Parameter	Setting
Nozzle Temperature	200-210°C
Bed Temperature	60°C
Layer Height	0.2mm
Infill Density	30-40%
Infill Pattern	Grid or Triangles
Wall Thickness	1.2mm (3 perimeters)
Top/Bottom Layers	4 layers
Print Speed	50-60 mm/s
Support	Yes, for overhangs

The nozzle temperature of 200-210°C provides good flow and layer adhesion for PLA. Higher temperatures improve layer bonding but may cause stringing or oozing.

The bed temperature of 60°C promotes first layer adhesion without excessive warping. This temperature is suitable for PLA on common build surfaces.

The layer height of 0.2mm provides good detail while maintaining reasonable print time. This is a common default that works well for most applications.

The infill density of 30-40% provides sufficient strength without excessive material usage. Higher infill increases strength but also weight and print time.

The infill pattern of grid or triangles provides good strength in all directions. Different patterns offer trade-offs between strength, print time, and material usage.

The wall thickness of 1.2mm (3 perimeters) creates strong outer shells. Multiple perimeters improve strength and appearance compared to single-perimeter walls.

The top and bottom layers of 4 layers provide solid surfaces. Adequate top and bottom layers prevent gaps and improve appearance.

The print speed of 50-60 mm/s balances quality with productivity. Slower speeds improve quality but increase print time.

Support structures are enabled for overhangs that would otherwise print in mid-air. Support is removed in post-processing.

9.2.3 Post-processing

After printing, several post-processing steps were performed to prepare the chassis for assembly.

Support structures were removed carefully to avoid damaging the part. Pliers, cutters, and files may be used for support removal.

Rough surfaces were sanded for smooth finish. Sanding improves appearance and removes layer lines that might affect component fit.

Mounting holes were cleaned with appropriate drill bits to ensure proper size and alignment. Printed holes may be slightly undersized due to material shrinkage.

All components were test-fitted before final assembly to verify dimensions and identify any issues. Test fitting is easier to perform before wiring and assembly.

Optional primer and paint can be applied for professional appearance. Painting improves appearance and can provide additional protection.

The resulting chassis provides a lightweight yet robust platform for the Fire Fighting Bot, with all components fitting precisely as designed in SolidWorks.

10. APPLICATIONS

The Fire Fighting Bot has numerous applications across various sectors where fire safety is a concern. The dual-mode operation and relatively low cost of the system make it suitable for a wide range of deployment scenarios. This chapter explores potential applications in industrial, commercial, and educational settings.

10.1 Industrial Applications

Industrial facilities often contain hazardous materials and complex machinery that can pose significant fire risks. The Fire Fighting Bot can be deployed in various industrial settings to enhance safety and reduce risks to human personnel.

Chemical plants represent high-risk environments where fires can release toxic materials and cause explosions. The Fire Fighting Bot can enter areas where hazardous chemicals are stored or processed, reducing risk to human firefighters. Its compact size allows access to spaces that may be difficult for larger equipment.

Warehouses and large storage facilities benefit from early fire detection and response. The Fire Fighting Bot can patrol storage areas continuously, detecting fires in their initial stages before they spread to large quantities of goods. Early intervention can prevent major losses and business interruptions.

Manufacturing units with machinery that may overheat or generate sparks can deploy the Fire Fighting Bot for continuous monitoring. The bot can respond immediately to fires caused by equipment failures, preventing damage to expensive machinery and production facilities.

Power plants face constant risk of electrical fires due to high-voltage equipment and extensive wiring. The Fire Fighting Bot can navigate electrical areas with appropriate precautions, providing early response to electrical fires before they damage critical infrastructure.

Oil and gas facilities handle flammable materials that require special firefighting considerations. The Fire Fighting Bot can operate in areas where flammable

materials are present, providing initial response while human firefighters prepare specialized equipment.

Data centers contain valuable equipment and data that can be destroyed by fire or water damage from sprinkler systems. The Fire Fighting Bot's targeted water delivery can suppress fires with minimal water usage, reducing damage to sensitive equipment.

The industrial applications leverage the Fire Fighting Bot's ability to operate autonomously or under remote control in hazardous environments. By reducing the need for human presence in dangerous areas, the bot protects worker safety while maintaining fire protection capabilities.

10.2 Commercial Applications

Commercial buildings and facilities can benefit from autonomous firefighting systems that provide protection without requiring dedicated staff. The Fire Fighting Bot offers an affordable solution for fire safety in various commercial settings.

Shopping malls are large public spaces where quick response is critical for public safety. The Fire Fighting Bot can patrol common areas and respond immediately to fires in food courts, retail spaces, or storage areas. Rapid response protects shoppers and minimizes business disruption.

Office buildings may be unoccupied during off-hours when fires can start unnoticed. The Fire Fighting Bot provides continuous monitoring and immediate response even when the building is empty. Remote notification through Telegram keeps facility managers informed of any incidents.

Hotels and restaurants face fire risks from kitchen equipment, heating systems, and guest activities. The Fire Fighting Bot can patrol kitchen areas, mechanical rooms, and storage spaces to detect and suppress fires before they spread to guest areas.

Parking garages are enclosed spaces where vehicle fires can spread quickly and produce toxic smoke. The Fire Fighting Bot can navigate parking structures to detect and suppress vehicle fires, protecting other vehicles and the structure itself.

Theaters and stadiums are large venues with high occupancy that require effective fire protection. The Fire Fighting Bot can patrol backstage areas, concession stands, and storage spaces where fires might start away from public view.

Commercial applications benefit from the Fire Fighting Bot's dual-mode operation. During business hours, manual mode allows security staff to control the bot for specific situations. After hours, automatic mode provides autonomous monitoring and response.

10.3 Educational Applications

The Fire Fighting Bot serves as an excellent educational tool for teaching robotics, electronics, programming, and engineering principles. Its comprehensive nature covers multiple disciplines in a single, engaging project.

Engineering education benefits from practical demonstration of robotics, sensors, IoT, and control systems. Students can study each subsystem in detail, understanding how mechanical, electronic, and software components work together. The project provides hands-on experience with industry-relevant technologies.

As a research platform, the Fire Fighting Bot provides a base for developing advanced algorithms and testing new sensors. Researchers can modify the basic design to explore machine learning for fire detection, advanced navigation algorithms, or multi-robot coordination.

Robotics competitions and hackathons often feature firefighting challenges. The Fire Fighting Bot provides a competitive platform that can be enhanced and customized for specific competition rules. Students gain experience in iterative design and competitive engineering.

STEM learning for students introduces programming, electronics, and 3D printing in an engaging context. The firefighting theme motivates students while the

hands-on construction develops practical skills. Students can start with basic functionality and add features as they learn.

Project-based learning covers multiple engineering disciplines in a comprehensive project. Students experience the complete engineering lifecycle from design through construction, testing, and iteration. This holistic approach develops skills that transfer to professional engineering practice.

The modular design and well-documented code make the Fire Fighting Bot an ideal platform for students to learn and experiment with various robotics concepts. Each subsystem can be studied independently before integration into the complete system.

Educational institutions can deploy Fire Fighting Bots as demonstration platforms for outreach and recruitment. The engaging nature of firefighting robots attracts interest in engineering programs and demonstrates practical applications of classroom learning.

11. ADVANTAGES AND LIMITATIONS

Understanding both the strengths and limitations of the Fire Fighting Bot is essential for proper deployment and future development. This chapter provides a balanced analysis of the system's capabilities, identifying areas where it excels and areas where improvements are needed.

11.1 Advantages

11.1.1 Safety Benefits

The primary advantage of the Fire Fighting Bot is the protection of human firefighters from dangerous situations. By sending a robot instead of a person into hazardous environments, the risk of injury or death is eliminated. This protection is invaluable in situations involving structural collapse, toxic smoke, or extreme heat.

The ability to operate 24/7 without fatigue or need for breaks ensures continuous protection. Unlike human firefighters who require rest, the Fire Fighting Bot can maintain vigilance indefinitely as long as power is available. This continuous operation is particularly valuable for facilities without full-time fire staff.

Access to hazardous environments that would be unsafe for humans expands firefighting capabilities. The robot can enter areas with toxic atmospheres, radiation, or structural instability that would prevent human entry. This access enables firefighting in previously unreachable locations.

Remote operation through Telegram integration allows triggering from a safe distance. Operators can initiate firefighting missions without entering the hazard zone, maintaining personal safety while addressing the emergency.

11.1.2 Economic Benefits

The low cost of components makes the Fire Fighting Bot affordable for organizations with limited budgets. Unlike commercial firefighting robots that cost hundreds of thousands of dollars, this project uses inexpensive, readily

available parts. This affordability democratizes access to robotic firefighting technology.

3D printed chassis manufacturing provides low-cost production with design flexibility. Changes to the design can be implemented quickly without expensive tooling or manufacturing setup. This flexibility supports rapid iteration and customization.

Reduced insurance costs may result from early fire detection and response. Insurance providers may offer discounts for facilities with enhanced fire protection systems. The Fire Fighting Bot provides documented, automated response that may qualify for such benefits.

Minimal maintenance requirements reduce ongoing operating costs. The simple mechanical design has few wear parts, and the electronic components are durable and easily replaced if needed.

11.1.3 Technical Benefits

Dual mode operation provides flexibility for different scenarios. Manual control enables precise operation in complex situations, while automatic mode provides immediate response without human intervention. Users can select the appropriate mode for each situation.

IoT integration through Telegram connectivity enables remote monitoring and control. Users can trigger missions, receive status updates, and monitor progress from anywhere with internet access. This connectivity extends the robot's utility beyond local operation.

Fast response to fire detection without human delay improves effectiveness. The robot reacts immediately to sensor inputs, beginning approach within milliseconds of detection. This speed can make the difference between containing a small fire and facing a major blaze.

Consistent performance without stress or panic ensures reliable operation in emergencies. Unlike humans who may experience stress or fear in dangerous situations, the robot maintains consistent behavior regardless of circumstances.

The modular design enables easy modification and upgrade of individual components. New sensors, actuators, or communication modules can be added without redesigning the entire system. This modularity supports continuous improvement and customization.

The educational value as a platform for learning robotics and embedded systems extends the project's impact beyond its immediate firefighting function. Students and hobbyists can learn from and build upon the design.

11.1.4 Operational Benefits

Autonomous operation without constant human supervision reduces labor requirements. Once deployed, the robot can monitor and respond to fires without requiring dedicated operators. This autonomy is valuable for facilities with limited staff.

Compact size enables navigation through narrow spaces that larger equipment cannot access. The robot can enter confined areas, pass through doorways, and maneuver in cluttered environments. This mobility extends firefighting reach.

Easy deployment with simple setup and calibration process reduces time to operational status. The robot can be placed in service quickly without extensive configuration or training.

Real-time monitoring through Telegram provides visibility into robot operation. Operators can track mission progress, verify system status, and receive alerts for important events.

11.2 Limitations

11.2.1 Technical Limitations

The limited range of flame sensors (typically 1-2 meters) restricts detection capability. Fires beyond this range may not be detected until the robot happens to approach them. This limitation requires the robot to search actively rather than monitoring large areas from a fixed position.

Environmental sensitivity affects performance under varying conditions. Ambient light, smoke, and temperature can interfere with sensor readings, potentially causing missed detections or false alarms. Calibration for specific environments helps but cannot eliminate all sensitivity.

Limited water capacity restricts extinguishing capability. The robot can only carry a few hundred milliliters of water, sufficient for small fires but inadequate for larger blazes. Multiple trips or refills may be needed for complete suppression.

Battery life limits operating time, especially with Wi-Fi active. The ESP32's wireless communication consumes significant power, reducing the time available for firefighting missions. Larger batteries extend runtime but increase weight and cost.

Basic obstacle avoidance may not handle complex terrain or obstacles. The current design lacks sophisticated navigation sensors, limiting its ability to navigate around obstacles or rough terrain.

Wi-Fi dependency for automatic mode requires network availability. Areas without Wi-Fi coverage cannot support automatic mode operation, limiting deployment options.

11.2.2 Operational Limitations

Effectiveness limited to small to medium fires constrains utility. The robot is not designed to fight large, established fires that require professional firefighting equipment and techniques.

Difficulty with multiple simultaneous fire sources may reduce effectiveness in complex scenarios. The robot's simple navigation algorithm focuses on a single fire and may not handle situations with multiple fire locations optimally.

Manual intervention required for water refill and battery charging limits autonomy. The robot cannot operate continuously without human assistance for resource replenishment.

Telegram API rate limits and potential service disruptions may affect reliability. Excessive messaging or Telegram service outages could impair communication between the robot and operators.

11.2.3 Environmental Limitations

Electronics may malfunction in extreme heat beyond the operating temperature range. While the ESP32 is rated for industrial temperatures, prolonged exposure to fire heat can damage components.

The robot is not waterproof and may be damaged by water or moisture. While designed to spray water, the electronics are not protected against water exposure. Splashes or submersion could cause failures.

Surface requirements for navigation limit deployment options. The robot requires relatively flat, smooth surfaces for effective movement. Rough terrain, stairs, or obstacles may prevent operation.

Flame sensor performance varies with ambient light conditions. Bright sunlight or artificial lighting may interfere with fire detection, requiring careful calibration for specific environments.

11.3 Mitigation Strategies

Many of the identified limitations can be addressed through design improvements and operational practices. These mitigation strategies provide pathways for enhancing the Fire Fighting Bot's capabilities.

Using higher quality flame sensors with longer detection range would extend the robot's detection capability. Advanced sensors can detect fires at greater distances, enabling earlier response and wider area coverage.

Implementing ultrasonic or infrared sensors for obstacle detection and avoidance would improve navigation in complex environments. These sensors would enable the robot to detect and navigate around obstacles autonomously.

Adding a larger water tank or considering alternative extinguishing agents would extend extinguishing capability. Foam or dry chemical agents might provide more effective suppression for certain fire types.

Using heat-resistant materials and enclosures for high-temperature environments would improve durability. Specialized materials can protect electronics from heat damage, extending operational range.

Implementing automatic return-to-base functionality for recharging would improve autonomy. The robot could return to a charging station when battery is low, enabling continuous operation with minimal human intervention.

Adding fallback communication methods such as SMS or local server connectivity would improve reliability. Multiple communication paths ensure connectivity even if one method fails.

Implementing waterproofing measures for critical components would improve reliability in wet conditions. Sealed enclosures and conformal coating can protect electronics from moisture damage.

12. FUTURE ENHANCEMENTS

The Fire Fighting Bot project provides a solid foundation for further development. This chapter explores potential enhancements that could improve the system's capabilities and expand its applications. These enhancements range from incremental improvements to transformative additions that would significantly expand functionality.

12.1 Sensor Upgrades

12.1.1 Advanced Fire Detection

Future versions could incorporate more sophisticated fire detection technologies that improve detection accuracy, range, and reliability. These advanced sensors would enhance the robot's ability to detect fires under various conditions.

Thermal imaging cameras provide visual heat maps for precise fire location. Unlike single-point infrared sensors, thermal cameras capture temperature distributions across a scene, enabling more accurate fire detection and tracking. Thermal imaging can detect fires through smoke and identify hot spots that may not be visible flames.

Smoke detectors can detect smoke before flames become visible. Early smoke detection enables response before fires grow large, potentially preventing significant damage. Smoke detection complements flame detection for comprehensive fire monitoring.

Gas sensors can detect combustible gases that may indicate fire risk. Detection of gas leaks or combustion byproducts provides early warning of potential fires. Gas sensing is particularly valuable in industrial environments with chemical hazards.

Multi-spectral sensors combine infrared, visible, and ultraviolet detection for comprehensive fire characterization. Different wavelengths provide different information about fires, and combining them improves detection reliability. Multi-spectral sensing can distinguish between different fire types and sources.

Temperature sensors can monitor ambient temperature changes that may indicate developing fires. Distributed temperature sensing provides area coverage beyond point sensors. Temperature trends can indicate fire growth or spread patterns.

12.1.2 Navigation Sensors

Enhanced navigation capabilities through additional sensors would improve the robot's ability to operate in complex environments. These sensors would enable more sophisticated autonomous behaviors.

Ultrasonic sensors provide obstacle detection and distance measurement. These sensors emit sound pulses and measure echo return time to determine distances. Ultrasonic sensing is effective for detecting obstacles regardless of lighting conditions.

Infrared distance sensors provide proximity detection for walls and objects. These sensors measure reflected infrared light to determine distances. IR sensors are compact and inexpensive, making them suitable for multiple placement around the robot.

LiDAR enables precise mapping and navigation in complex environments. LiDAR systems scan the surroundings with laser light, creating detailed 3D maps. This capability would enable advanced path planning and obstacle avoidance.

IMU (Inertial Measurement Unit) provides orientation and movement tracking. Combining accelerometer, gyroscope, and magnetometer data, an IMU tracks the robot's position and orientation. This information improves navigation accuracy and enables dead reckoning when other sensors are unavailable.

Encoders on wheel shafts provide precise wheel rotation measurement for accurate positioning. By counting wheel rotations, the robot can estimate distance traveled and calculate position. Encoder data complements other navigation sensors for improved accuracy.

12.2 Communication Capabilities

12.2.1 Enhanced Wireless Communication

Adding advanced communication capabilities would enable better remote monitoring and control. These enhancements would expand the robot's operational range and reliability.

5G modules provide high-speed, low-latency communication for real-time control. 5G networks offer significantly higher bandwidth and lower delay than current Wi-Fi, enabling video streaming and responsive remote control. This capability would support teleoperation from anywhere with 5G coverage.

Mesh networking enables multiple robots to communicate and coordinate. In a mesh network, each robot can relay messages for others, extending communication range beyond direct radio contact. This capability supports collaborative firefighting with multiple robots.

Satellite communication enables operation in remote areas without cellular coverage. For offshore platforms, wilderness facilities, or disaster zones, satellite links provide connectivity where other methods fail.

LoRaWAN provides long-range, low-power communication for large facilities. LoRaWAN networks can cover square kilometers with minimal infrastructure, making them suitable for industrial campuses and warehouses.

12.2.2 Cloud Integration

Cloud connectivity would enable advanced capabilities that leverage internet-scale computing and storage. These capabilities would transform the Fire Fighting Bot from a standalone device to a connected system component.

Real-time video streaming provides live camera feed from the robot. Operators could see what the robot sees, enabling better situational awareness and decision-making. Video streaming would support both automatic and manual modes.

Sensor data logging records and analyzes sensor readings over time. Historical data enables trend analysis, performance optimization, and predictive

maintenance. Cloud storage provides virtually unlimited capacity for data accumulation.

Remote firmware updates enable over-the-air updates for bug fixes and improvements. New features could be deployed without physical access to the robot, reducing maintenance costs and improving responsiveness.

Fleet management enables monitoring and control of multiple robots from a central dashboard. Large facilities could deploy multiple Fire Fighting Bots with coordinated coverage and shared management.

AI-powered analytics apply machine learning to fire pattern recognition. Analysing historical fire data could identify risk factors, predict fire likelihood, and optimize response strategies.

12.3 Intelligence and Automation

12.3.1 Machine Learning

Machine learning algorithms could enhance the robot's capabilities by enabling adaptive behaviour and improved decision-making. These algorithms would learn from experience to improve performance over time.

Fire pattern recognition learns to distinguish between real fires and false positives. By analysing sensor data patterns, machine learning models can identify characteristics that indicate actual fires versus similar but harmless sources. This capability would reduce false alarms and improve response accuracy.

Path optimization learns optimal routes through frequently visited areas. By analyzing successful navigation patterns, the robot can identify efficient paths that minimize response time. This learning improves performance in familiar environments.

Predictive maintenance predicts component failures before they occur. By monitoring sensor data and operational patterns, machine learning can identify signs of impending failure. Predictive maintenance reduces downtime and repair costs.

Adaptive behavior adjusts behavior based on environment and experience. The robot could learn which strategies work best in different situations and adapt accordingly. This adaptability improves performance across varying conditions.

12.3.2 Swarm Intelligence

Multiple robots could work together as a coordinated team, achieving objectives that exceed individual capabilities. Swarm intelligence enables emergent behaviors from simple individual rules.

Distributed coverage enables multiple robots to patrol larger areas. Each robot covers a portion of the total area, with coordination ensuring complete coverage. This approach scales firefighting capability with the number of robots deployed.

Collaborative extinguishing enables multiple units to attack large fires simultaneously. Coordinated water delivery from multiple angles improves suppression effectiveness. Swarm coordination ensures efficient resource utilization.

Communication networks enable robots to share information about fire locations. When one robot detects a fire, others can respond to the same location or adjust their patrol patterns. Information sharing improves overall system effectiveness.

Load balancing efficiently distributes tasks among available units. The swarm can dynamically assign robots to fires based on proximity, capability, and availability. This optimization maximizes response effectiveness.

12.4 Mechanical Improvements

12.4.1 Mobility Enhancements

Improved mobility would expand the robot's operational range to include more challenging environments. These enhancements would enable operation in scenarios beyond the current design's capabilities.

All-terrain wheels handle rough surfaces, stairs, and obstacles. Specialized wheel designs with deep treads, flexible materials, or articulated mounts improve

traction and obstacle handling. All-terrain capability extends deployment options to outdoor and industrial environments.

Tracked drives provide better traction on slippery or uneven surfaces. Tracks distribute weight over a larger area, reducing ground pressure and improving grip. Tracked vehicles can navigate soft ground and climb obstacles that wheeled vehicles cannot.

Articulated arms reach fires in elevated or confined spaces. A robotic arm could extend the nozzle into spaces the robot cannot enter, such as behind equipment or above ceiling tiles. Articulated manipulation significantly extends reach.

Waterproof designs enable operation in wet conditions or shallow water. Sealed enclosures and corrosion-resistant materials protect components from moisture damage. Waterproofing is essential for outdoor deployment and flood scenarios.

12.4.2 Extinguishing System Upgrades

Enhanced extinguishing capabilities would improve fire suppression effectiveness. These upgrades would expand the types of fires the robot can address and improve suppression efficiency.

Multiple extinguishing agents including water, foam, CO2, or dry chemical enable addressing different fire classes. Different fires require different suppression approaches, and agent selection improves effectiveness. Automated agent selection based on fire characteristics would optimize suppression.

Adjustable nozzles control spray pattern and range. Narrow streams provide long reach for distant fires, while wide sprays cover large areas up close. Adjustable nozzles enable optimization for each fire scenario.

Larger capacity tanks extend operation with bigger water supplies. More water enables longer extinguishing duration or multiple fire attacks without refilling. Capacity must be balanced against weight and mobility impacts.

Automatic refill systems enable return to base station for automatic water refill. The robot could detect low water levels, return to a filling station, and resume

operation without human intervention. This capability would significantly extend autonomous operation time.

12.5 Integration with Smart Systems

Integration with building management systems could provide comprehensive fire safety that coordinates multiple systems. This integration would enable proactive fire prevention as well as responsive suppression.

Building automation interfaces with HVAC, lighting, and security systems. Coordinated responses could shut down ventilation to prevent fire spread, activate emergency lighting, and unlock exit routes. Integration multiplies the value of each individual system.

Fire alarm integration coordinates with existing fire alarm systems. The robot could respond to alarm triggers from smoke detectors or pull stations, providing immediate suppression while human responders are en route.

Emergency response automatically notifies emergency services when fires are detected. Integration with dispatch systems ensures rapid professional response while the robot provides initial suppression.

Evacuation assistance guides building occupants to safe exits. The robot could use speakers or displays to provide directions, or physically lead people along safe routes. This capability adds life safety to property protection.

Smart home integration connects with home automation platforms. Residential deployments could integrate with popular platforms, enabling voice control, automated responses, and unified monitoring.

13. CONCLUSION

The Fire Fighting Bot with Dual Mode Operation represents a successful integration of embedded systems, IoT communication, robotics, and fire safety principles. This project demonstrates how relatively simple and affordable components can be combined to create a functional autonomous system that effectively addresses a real-world safety challenge.

13.1 Summary of Achievements

The project has achieved significant objectives that demonstrate both technical capability and practical utility. These achievements validate the design approach and provide a foundation for future development.

Designed a robust chassis using SolidWorks and manufactured it using 3D printing technology with PLA material. The chassis accommodates all components while maintaining stability and ease of assembly. The CAD-based design process enabled virtual verification before physical construction.

Implemented a dual-mode operation system with Manual mode via Bluetooth RC app and Automatic mode via Telegram-triggered missions. Both modes function reliably, providing flexibility for different operational scenarios. The dual-mode capability distinguishes this project from single-mode alternatives.

Integrated flame sensors for directional fire detection and servo motor for precise nozzle positioning. The sensor system reliably detects fires and provides directional information for navigation. The servo positioning improves extinguishing effectiveness by ensuring accurate water delivery.

Created an effective extinguishing mechanism using a water pump system with proper power supply management using buck converter. The extinguishing system delivers adequate water for small fire suppression while managing power efficiently. The buck converter provides stable power with high efficiency.

Demonstrated the practical application of embedded systems and IoT in solving real-world fire safety problems. The completed project shows that effective

firefighting robots can be built using affordable, readily available components. This demonstration opens possibilities for wider adoption of robotic firefighting.

By utilizing technologies such as the ESP32 microcontroller, flame sensors, motor drivers, and wireless communication through Telegram, the system showcases the practical implementation of modern engineering concepts in a compact and efficient design. The integration of these diverse technologies demonstrates the power of modern embedded systems.

The dual-mode functionality, consisting of both manual control via Bluetooth and automatic operation through IoT-based communication, enhances the flexibility and usability of the system. It allows users to operate the robot in different environments based on requirements, ensuring both precision and automation.

Furthermore, this project highlights the importance of automation in reducing human risk in dangerous situations such as fire accidents. It provides a cost-effective and scalable solution that can be further enhanced with advanced technologies like artificial intelligence, image processing, and cloud-based monitoring systems.

Overall, the Fire Fighting Bot not only serves as an innovative engineering solution but also as a valuable learning platform that demonstrates the real-world application of embedded systems, robotics, and IoT in improving safety and efficiency.

13.2 Key Learnings

The development of this project provided valuable insights into several technical areas. These learnings inform future development and demonstrate the educational value of hands-on projects.

Embedded systems programming experience with ESP32 programming, sensor interfacing, and motor control provided practical skills applicable to many other projects. The ESP32 platform offers powerful capabilities that enable sophisticated applications.

3D modeling and printing skills developed through SolidWorks design and FDM 3D printing enable rapid prototyping of mechanical components. These skills bridge the gap between digital design and physical realization.

IoT communication experience with Wi-Fi connectivity, HTTP protocols, and API integration demonstrates how embedded devices can interact with cloud services. This knowledge is increasingly valuable as more devices become connected.

System integration understanding of how mechanical, electronic, and software components work together is essential for complex projects. Successful integration requires attention to interfaces, timing, and interactions between subsystems.

Problem-solving skills developed through debugging and troubleshooting hardware and software issues are transferable to many technical challenges. Systematic approaches to problem identification and resolution improve efficiency and outcomes.

13.3 Significance

The Fire Fighting Bot project holds significance in multiple dimensions that extend beyond its immediate technical achievements.

From an educational perspective, it provides a comprehensive learning platform for students interested in robotics, IoT, and embedded systems. The project covers fundamental concepts including sensor interfacing, motor control, IoT communication, 3D modeling,

From a practical perspective, it demonstrates the potential for low-cost automation in fire safety applications. While the current prototype has limitations, it serves as a proof of concept for more sophisticated systems. The demonstration that effective firefighting robots can be built affordably may inspire further development and adoption.

From a societal perspective, it contributes to the ongoing effort to protect human lives through technological innovation. Firefighting robots have the potential to

reduce injuries and fatalities among firefighters while improving response times. Every life saved through such technology represents a significant achievement.

The project also demonstrates the democratization of advanced technology through open-source platforms and affordable components. Students, hobbyists, and small organizations can build upon this work, spreading the benefits of robotics more broadly than proprietary commercial solutions.

13.4 Future Outlook

The field of firefighting robotics continues to evolve rapidly. Advances in artificial intelligence, sensor technology, and materials science will enable increasingly capable autonomous systems. The Fire Fighting Bot project provides a foundation upon which these future developments can be built.

As technology becomes more accessible and affordable, systems like the Fire Fighting Bot could become common fixtures in industrial facilities, commercial buildings, and eventually residential settings. The integration of wireless communication, machine learning, and advanced sensors will transform simple robots into intelligent safety systems.

Future developments may include enhanced sensors for improved fire detection, advanced navigation for complex environments, and collaborative operation with multiple robots. Machine learning will enable adaptive behavior that improves with experience. Cloud connectivity will enable centralized monitoring and control of distributed robot fleets.

The trend toward smart buildings and IoT will create opportunities for firefighting robots to integrate with building management systems. Coordinated responses involving robots, alarms, ventilation systems, and emergency services will provide comprehensive fire safety.

Research in firefighting robotics will continue to push the boundaries of what is possible. Academic and industry research will develop new technologies that eventually become available for practical applications. The Fire Fighting Bot

project contributes to this ecosystem by demonstrating practical implementations of research concepts.

13.5 Final Remarks

The successful completion of this project demonstrates that effective firefighting robots can be built using readily available components and open-source platforms. The ESP32 ecosystem, in particular, has enabled powerful IoT capabilities at an affordable price point, making such projects accessible to students, hobbyists, and researchers.

As we look toward the future, the lessons learned from this project will inform the development of more advanced systems. The combination of practical experience gained and theoretical knowledge applied positions this work as a valuable contribution to the field of fire safety robotics.

In conclusion, the Fire Fighting Bot with Dual Mode Operation successfully demonstrates the potential of embedded systems and IoT in addressing critical safety challenges. Through continued development and innovation, such systems will play an increasingly important role in protecting lives and property from the devastating effects of fire.

The project serves as a testament to what can be achieved with determination, creativity, and modern technology. It shows that meaningful contributions to important problems can come from individual efforts and modest resources. The Fire Fighting Bot stands as an example of how engineering can serve society and inspire future generations of technologists.

14. REFERENCES

The following references were consulted during the development of this project and provide additional information on the technologies and concepts employed.

1. Mazidi, M. A., McKinlay, R. D., & Causey, D. (2012). PIC Microcontroller and Embedded Systems: Using Assembly and C for PIC18. Pearson Education.

Content Used: Microcontroller fundamentals, embedded system concepts, interfacing techniques

2. Monk, S. (2013). Programming Arduino: Getting Started with Sketches. McGraw-Hill Education.

Content Used: Arduino programming logic, sensor interfacing, control structures

3. Banzi, M., & Shiloh, M. (2014). Make: Getting Started with Arduino. Maker Media.

Content Used: Arduino basics, hardware-software integration, prototyping concepts

4. Barrett, S. F., & Pack, D. J. (2005). Microcontrollers Fundamentals for Engineers and Scientists. Morgan & Claypool Publishers.

Content Used: Data processing, input-output operations, system design principles

5. Iovine, J. (2012). Robots, Androids, and Animatrons. McGraw-Hill Education.

Content Used: Robotics concepts, movement mechanisms, automation basics

6. Murphy, R. R., & Burke, J. L. (2010). Human-Robot Interaction in Rescue Robotics. IEEE Transactions on Systems, Man, and Cybernetics.

Content Used: Firefighting robots, rescue operations, real-world robotics applications

7. National Fire Protection Association (NFPA). (2021). Fire Loss in the United States. NFPA Research.

Content Used: Fire safety statistics, problem background, motivation

8. Arduino Documentation (2023). https://docs.arduino.cc

Content Used: Arduino Uno specifications, IDE usage, hardware interfacing

9. Espressif Systems (2023). https://www.espressif.com

Content Used: ESP32 features, Wi-Fi capabilities, GPIO and system architecture

10. Telegram Bot API (2023). https://core.telegram.org/bots/api

Content Used: Telegram integration, API communication, messaging system

11. Siegwart, R., & Nourbakhsh, I. R. (2004). Introduction to Autonomous Mobile Robots. MIT Press.

Content Used: Navigation algorithms, robot movement logic

12. Thrun, S., Burgard, W., & Fox, D. (2005). Probabilistic Robotics. MIT Press.

Content Used: Autonomous decision making, sensor-based navigation

13. Flame Sensor Datasheet (KY-026). Keyes Corporation.

Content Used: Flame detection principle, IR sensing, sensor specifications

14. L298N Motor Driver Datasheet. STMicroelectronics.

Content Used: Motor control, H-bridge working, circuit design

15. Arduino IDE Documentation (2023). https://www.arduino.cc/en/software

Content Used: Software development environment, coding and uploading process